Machine Learning

Exam Notes

Short, precise, to-the-point theory for NBCA-601

10 chapters · 24 hours · exam ready



Companion to the textbook *Machine Learning: A Friendly Course for BCA Students*

Definitions · Key points · Formulas · Algorithms · Diagrams to draw · Numericals · Important questions

How to Finish the Syllabus in 24 Hours

These notes contain **only what you write in the exam**: definitions, key points, formulas, algorithm steps, comparison tables, diagrams to draw and the standard numericals. No code and no long stories: for those, use the main textbook.

The 24-hour plan

Session	Chapter	Hours	Must be able to do
1	1 Introduction to ML	2.0	define ML, types of ML, challenges, overfitting
2	2 Statistical learning	2.5	loss, risk, bias–variance, CV, metrics, standard error
3	3 k-NN, trees, Naive Bayes	2.5	k-NN by hand, entropy and gain, Bayes by hand
4	4 Linear models, GLM, SVM	2.5	least-squares line, R^2, sigmoid, log-loss, margin
5	5 Ensemble learning	2.5	voting, bagging, random forest, AdaBoost, stacking
6	6 SVM and Naive Bayes	2.0	kernels, C and γ, SVR, NB variants
7	7 Clustering	2.5	K-Means by hand, DBSCAN, GMM and EM
8	8 Dimensionality reduction	2.0	curse of dimensionality, PCA by hand
9	9 Neural networks with Keras	2.5	perceptron, MLP, activations, backpropagation
10	10 TensorFlow and data	1.5	tensors, `tf.data`, preprocessing layers
11	Last-hour sheet + mock test	1.5	formulas, comparisons, one full question paper
	Total	**24.0**	

Suggested split: three days of 8 hours (Day 1: Ch. 1–3 + half of 4; Day 2: rest of 4 to 7; Day 3: 8–10 and revision), or six days of 4 hours.

How to study each chapter (in this order)

1. **Definitions** (blue boxes): read aloud, then write each one from memory once. They are the 2-mark answers.
2. **Key points and tables**: these are the body of 5-mark answers.
3. **Diagrams to draw** (green boxes): draw each one once on paper.
4. **Numerical patterns** (orange boxes): cover the solution and solve it yourself.
5. **Important questions**: answer the 2-mark ones orally; outline two of the 10-mark ones.

Answer-writing formula

2 marks	definition (2–3 lines) + one formula or example
5 marks	definition + diagram + 4–5 points + small example
10 marks	intro + definition + steps/algorithm + diagram + numerical + advantages/disadvantages + applications + conclusion
Numerical	Given → formula → substitution → step-by-step working → answer with its meaning

Exam focus

The boxes mean: **Definition** = write exactly; **Exam focus** = most-asked point; **Diagram to draw** = practise once; **Numerical pattern** = solve once without looking. The line **Advantages / Disadvantages / Applications** under each method is ready to copy at the end of a long answer.

Contents

How to Finish the Syllabus in 24 Hours i

1 Introduction to Machine Learning 1

1.1 AI, ML and Deep Learning . . . 1
1.2 Types of Machine Learning Systems . . . 1
1.2.1 By supervision . . . 1
1.2.2 Batch vs. online learning . . . 2
1.2.3 Instance-based vs. model-based learning . . . 3
1.3 Main Challenges of Machine Learning . . . 3
1.4 Testing and Validating . . . 4

2 Statistical Learning 5

2.1 Statistical Learning: the Basic Model . . . 5
2.2 Supervised vs. Unsupervised Learning . . . 5
2.3 Loss Functions, Training and Test Loss . . . 5
2.4 Risk and Its Estimation . . . 6
2.5 Trade-offs in Statistical Learning . . . 6
2.5.1 Bias–variance trade-off . . . 7
2.5.2 Other trade-offs . . . 7
2.6 Classification Metrics . . . 7
2.7 Sampling Distribution of an Estimator . . . 8

3 Distance-Based Methods, Decision Trees and Naive Bayes 9

3.1 Distance-Based Methods . . . 9
3.2 k-Nearest Neighbours (k-NN) . . . 9
3.3 Decision Trees . . . 10
3.4 Naive Bayes . . . 11

4 Linear Models, GLMs and SVM Basics 13

4.1 Linear Regression . . . 13
4.1.1 Gradient descent . . . 13
4.1.2 Polynomial regression and regularisation . . . 14
4.2 Logistic Regression . . . 14
4.3 Generalised Linear Models (GLMs) . . . 15
4.4 Support Vector Machines (basics) . . . 15
4.5 Binary Classification . . . 16

5 Ensemble Learning 17

5.1 Voting Classifiers . . . 17
5.2 Bagging and Pasting . . . 17
5.3 Random Forests . . . 18

5.4 Boosting . . . 18
5.4.1 AdaBoost (Adaptive Boosting) . . . 18
5.4.2 Gradient boosting . . . 19
5.5 Stacking . . . 19

6 Support Vector Machines and Naive Bayes Classifiers 21
6.1 Linear SVM Classification . . . 21
6.2 Non-linear SVM Classification . . . 21
6.3 SVM Regression (SVR) . . . 22
6.4 Naive Bayes Classifiers . . . 22

7 Clustering 24
7.1 K-Means . . . 24
7.1.1 Choosing k and better initialisation . . . 24
7.1.2 Limits of K-Means . . . 25
7.2 Uses of Clustering . . . 25
7.3 DBSCAN . . . 25
7.4 Gaussian Mixture Models (GMM) . . . 26

8 Dimensionality Reduction 28
8.1 The Curse of Dimensionality . . . 28
8.2 Main Approaches . . . 28
8.3 Principal Component Analysis (PCA) . . . 29

9 Artificial Neural Networks with Keras 31
9.1 From Biological to Artificial Neurons . . . 31
9.2 The Perceptron . . . 31
9.3 Multilayer Perceptron (MLP) and Backpropagation . . . 32
9.4 Activation Functions . . . 32
9.5 Implementing MLPs with Keras . . . 33
9.6 Training Deep Networks: Key Ideas . . . 33

10 TensorFlow: Loading and Preprocessing Data 35
10.1 TensorFlow . . . 35
10.2 Loading Data: the `tf.data` API . . . 35
10.2.1 The TFRecord format . . . 36
10.3 Preprocessing Data . . . 36

Last-Hour Revision Sheet 38

1. Introduction to Machine Learning

Study time: 2 hours

1.1 AI, ML and Deep Learning

Definition: Artificial Intelligence (AI)

AI is the branch of computer science that builds machines able to perform tasks that normally need human intelligence, such as reasoning, understanding language, seeing and decision making.

Definition: Machine Learning (Arthur Samuel, 1959)

Machine Learning is the field of study that gives computers the ability to **learn from data without being explicitly programmed**.

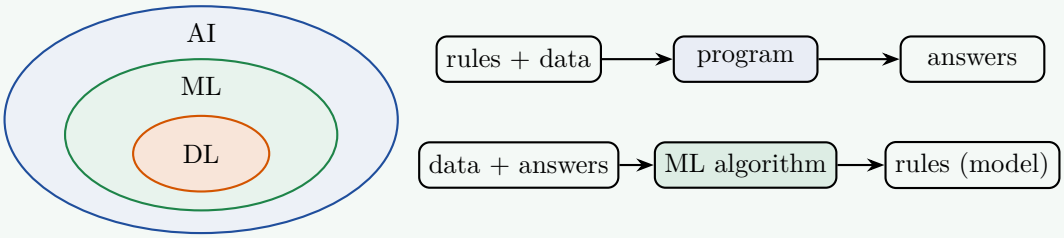
Definition: Machine Learning (Tom Mitchell, 1997)

A computer program is said to **learn** from experience E with respect to some task T and performance measure P, if its performance at T, as measured by P, **improves with experience** E.

Example (spam filter): T = classify e-mails as spam or not; E = e-mails labelled by users; P = percentage of e-mails classified correctly.

- **AI ⊃ ML ⊃ Deep Learning.** DL = ML with many-layered neural networks.
- **Traditional programming:** rules + data → answers. **ML:** data + answers → rules (model).
- **Use ML when:** rules are too many or unknown (speech, images), the problem changes over time (spam), or we want insight from large data (data mining).

Diagram to draw: AI, ML and DL as nested circles; traditional vs. ML



Applications: spam filtering, product recommendation, face recognition, speech assistants, fraud detection, medical diagnosis, self-driving cars, language translation, stock forecasting.

1.2 Types of Machine Learning Systems

ML systems are classified on three bases:

1. **Amount and type of supervision:** supervised, unsupervised, semi-supervised, reinforcement.
2. **Can it learn incrementally?** batch (offline) vs. online learning.
3. **How does it generalise?** instance-based vs. model-based learning.

1.2.1 By supervision

Definition: Supervised learning

The training data includes the desired outputs, called **labels**. The model learns a mapping from inputs to labels. **Classification** predicts a class (spam/not spam); **regression** predicts a number (house price).

Algorithms: k-NN, linear and logistic regression, SVM, decision trees, random forests, neural networks.

Definition: Unsupervised learning

The training data is **unlabelled**; the system finds structure (groups, patterns) by itself.

Tasks: clustering (K-Means, DBSCAN), dimensionality reduction (PCA), anomaly detection, association rules (Apriori: "bread ⇒ butter").

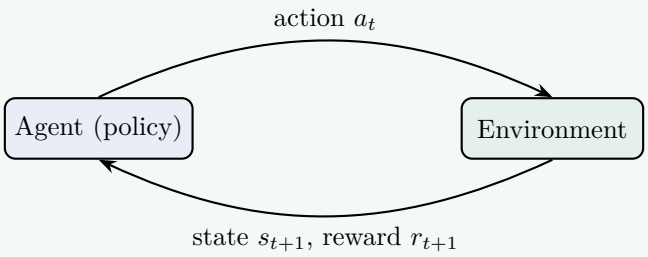
Definition: Semi-supervised learning

Learning from a **small amount of labelled** data plus a **large amount of unlabelled** data. Example: a photo app clusters faces (unsupervised), you name one face per person, and it labels all photos.

Definition: Reinforcement learning

An **agent** observes the **environment**, takes **actions** and receives **rewards** or penalties. It learns a **policy** (which action to take in each state) that maximises the total reward over time. Examples: game playing (AlphaGo), robots learning to walk.

Diagram to draw: Reinforcement learning loop



	Supervised	Unsupervised	Semi-supervised	Reinforcement
Data	labelled	unlabelled	few labels, many unlabelled	rewards from environment
Goal	predict label	find structure	use both	maximise reward
Feedback	correct answer	none	partial	delayed reward
Example	spam, price	customer groups	photo tagging	games, robots

1.2.2 Batch vs. online learning

	Batch (offline) learning	Online (incremental) learning
Training	on all data at once	on small mini-batches, continuously
New data	retrain from scratch	learn on the fly
Resources	large memory and time	small; suits huge or streaming data
Key parameter	—	**learning rate**: high = adapts fast but forgets
Risk	model becomes stale	bad data quietly degrades it
Example	monthly retrained recommender	stock prices, sensor streams

Out-of-core learning: online learning used to train on data too large for memory.

1.2.3 Instance-based vs. model-based learning

Definition: Instance-based learning

The system **memorises** the training examples and generalises to a new case by comparing it with stored examples using a **similarity measure**. Example: k-NN.

Definition: Model-based learning

The system builds a **model** (e.g. a line $\hat{y} = \theta_0 + \theta_1 x$) from the examples, by choosing parameters that minimise a **cost function**, and then uses the model to predict.

Model-based workflow: study the data → select a model → train it (minimise cost) → apply it to new cases (**inference**).

Numerical pattern: least-squares line

Data $x = (1, 2, 3, 4, 5)$ (size), $y = (15, 26, 29, 44, 46)$ (price). $\bar{x} = 3$, $\bar{y} = 32$.
$S_{xy} = \sum(x - \bar{x})(y - \bar{y}) = 34 + 6 + 0 + 12 + 28 = 80$, $S_{xx} = \sum(x - \bar{x})^2 = 10$.
$\theta_1 = S_{xy}/S_{xx} = 8$, $\theta_0 = \bar{y} - \theta_1 \bar{x} = 32 - 24 = 8$. **Model** $\hat{y} = 8 + 8x$; for $x = 6$: $\hat{y} = 56$.

1.3 Main Challenges of Machine Learning

Problems come from either **bad data** or a **bad algorithm (model)**.

1. **Insufficient training data:** most algorithms need thousands of examples; "data matters more than algorithms".
2. **Non-representative data:** training data must represent the new cases. *Sampling noise* (sample too small) and *sampling bias* (flawed sampling method).
3. **Poor-quality data:** errors, outliers, missing values. Clean, remove or fill them.
4. **Irrelevant features:** garbage in, garbage out. *Feature engineering* = feature selection + feature extraction + creating new features.
5. **Overfitting** the training data.
6. **Underfitting** the training data.

Definition: Overfitting

Overfitting occurs when the model fits the training data (including its noise) too closely, so it performs well on training data but **poorly on new data**. Cause: model too complex for the amount and noise of the data.

Remedies: simpler model (fewer parameters), more training data, reduce noise, **regularisation** (constrain the model), cross-validation, early stopping.

Definition: Underfitting

Underfitting occurs when the model is **too simple** to learn the underlying structure, so it performs poorly even on the training data.

Remedies: more powerful model, better features, reduce regularisation.

Diagram to draw: Underfit, good fit, overfit



	Underfitting	Overfitting
Model	too simple	too complex
Training error	high	very low
Test error	high	high
Bias / variance	high bias	high variance
Fix	more complex model, better features	more data, regularisation, simpler model

1.4 Testing and Validating

- Split data into **training set** (usually 80%) and **test set** (20%). The error on new cases is the **generalisation error**, estimated on the test set.
- Training error low but generalisation error high ⇒ overfitting.
- **Hyperparameter:** a parameter of the learning algorithm (not of the model), set *before* training, e.g. k in k-NN, regularisation strength. **Parameters** (weights) are learned.
- **Validation set (holdout validation):** part of the training set used to compare models and tune hyperparameters; the test set is used only once at the end.
- **Cross-validation:** rotate the validation part (see Chapter 2).
- **No Free Lunch theorem:** no model is best for every problem; you must try several.

Exam focus

Most asked: Mitchell's definition with a (T, P, E) example; types of ML with a table; overfitting vs. underfitting with a diagram; challenges of ML. Always give one real-life example for each type.

Important Questions

- **[2 marks]** Define machine learning. Give T, P, E for a spam filter.
- **[2 marks]** Differentiate between AI and ML.
- **[2 marks]** What is a hyperparameter? Give an example.
- **[2 marks]** What is semi-supervised learning?
- **[5 marks]** Compare supervised and unsupervised learning with examples.
- **[5 marks]** Explain batch and online learning.
- **[5 marks]** Explain instance-based and model-based learning.
- **[5 marks]** What is overfitting? How can it be avoided?
- **[10 marks]** Explain the types of machine learning systems in detail with examples and a diagram.
- **[10 marks]** Discuss the main challenges of machine learning.

2. Statistical Learning

Study time: 2.5 hours

2.1 Statistical Learning: the Basic Model

Definition: Statistical learning

Statistical learning is a set of tools for estimating the relationship f between input variables $X = (X_1, \ldots, X_p)$ (predictors, features) and an output Y (response):

$$Y = f(X) + \varepsilon,$$

where f is fixed but unknown and ε is a random error with mean zero, independent of X.

- **Why estimate f?** (i) **Prediction**: $\hat{Y} = \hat{f}(X)$ (treat $\hat{f}$ as a black box). (ii) **Inference**: understand *how* Y depends on each X_j.
- **Reducible error:** $[f(X) - \hat{f}(X)]^2$, can be reduced with a better model.
- **Irreducible error:** $\mathrm{Var}(\varepsilon) = \sigma^2$, cannot be removed by any model.

$$\mathbb{E}[(Y - \hat{Y})^2] = \underbrace{[f(X) - \hat{f}(X)]^2}_{\text{reducible}} + \underbrace{\sigma^2}_{\text{irreducible}}$$

How to estimate f:

- **Parametric methods:** assume a form (e.g. linear), then estimate a few parameters. Simple, but wrong form $\Rightarrow$ poor fit.
- **Non-parametric methods:** no fixed form (e.g. k-NN, splines). Flexible, but need much data and may overfit.

2.2 Supervised vs. Unsupervised Learning

- **Supervised:** for every x_i there is a response y_i; fit a model to predict y. Regression (quantitative y) or classification (qualitative y).
- **Unsupervised:** only x_i, no y_i. Goal: understand relationships between variables or observations, e.g. **cluster analysis**.
- **Semi-supervised:** y known for only some observations.

2.3 Loss Functions, Training and Test Loss

Definition: Loss function

A **loss function** $L(y, \hat{y})$ measures the penalty for predicting $\hat{y}$ when the true value is y.

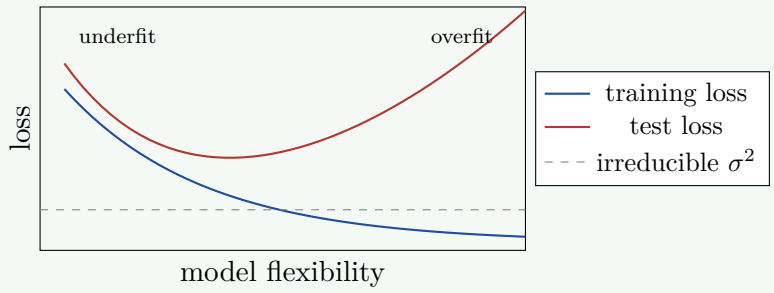
Loss	Formula	Used for
Squared loss	$(y - \hat{y})^2$	regression (MSE)
Absolute loss	$\lvert y - \hat{y} \rvert$	regression (MAE), robust to outliers
0–1 loss	$\mathbb{1}[y \neq \hat{y}]$	classification (error rate)
Log-loss	$-[y \ln p + (1-y)\ln(1-p)]$	probabilistic classification

Definition: Training loss and test loss

Training loss is the average loss on the data used to fit the model: $\frac{1}{n}\sum_{i=1}^{n} L(y_i, \hat{f}(x_i))$. **Test loss** is the average loss on new, unseen data. The test loss measures how well the model **generalises**.

- Training loss always falls as model **flexibility** increases.
- Test loss is **U-shaped**: it falls, reaches a minimum, then rises (overfitting).
- A small training loss does **not** guarantee a small test loss.

Diagram to draw: Training and test loss vs. flexibility



Numerical pattern: computing losses

$y = (3, 5, 2)$, $\hat{y} = (2.5, 5, 4)$. Errors $= (0.5, 0, -2)$.
MSE $= \frac{0.25+0+4}{3} = 1.417$; RMSE $= \sqrt{1.417} = 1.190$; MAE $= \frac{0.5+0+2}{3} = 0.833$.

2.4 Risk and Its Estimation

Definition: Risk and empirical risk

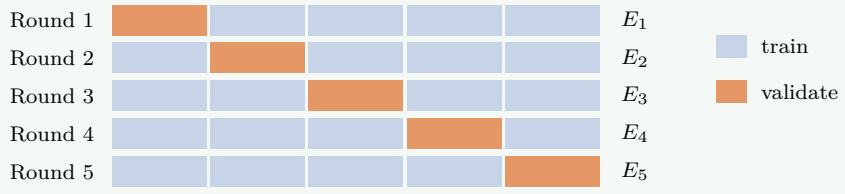
The **risk** of a model f is its **expected loss** over the whole population: $\mathcal{R}(f) = \mathbb{E}[L(Y, f(X))]$. Because the population is unknown, we use the **empirical risk** (average loss on a sample): $\hat{\mathcal{R}}(f) = \frac{1}{n}\sum_i L(y_i, f(x_i))$.

Estimating risk (test error):

1. **Validation-set approach:** split into training and validation parts. Simple, but the estimate varies a lot and uses less data.
2. **Leave-one-out CV (LOOCV):** train on $n-1$ points, test on 1, repeat n times. Low bias, but costly.
3. **k-fold cross-validation:** split data into k folds; each fold is the validation set once:

$$\mathrm{CV}_{(k)} = \frac{1}{k}\sum_{i=1}^{k} E_i \qquad (k = 5 \text{ or } 10).$$

Diagram to draw: 5-fold cross-validation



Numerical pattern: 5-fold CV

Fold errors $0.20, 0.25, 0.15, 0.22, 0.18$. $\mathrm{CV}_{(5)} = \frac{1.00}{5} = \mathbf{0.20}$.

2.5 Trade-offs in Statistical Learning

2.5.1 Bias–variance trade-off

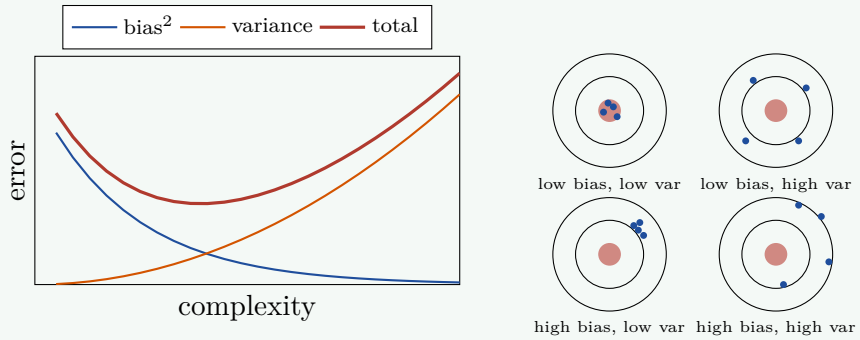
Definition: Bias and variance of a model

Bias is the error from approximating a complex real problem by a simpler model (systematic error). **Variance** is the amount by which $\hat{f}$ would change if it were trained on a different training set (sensitivity to data).

$$\mathbb{E}[(y_0 - \hat{f}(x_0))^2] = \text{Bias}\left[\hat{f}(x_0)\right]^2 + \text{Var}\left[\hat{f}(x_0)\right] + \sigma^2$$

- More flexible model $\Rightarrow$ bias $\downarrow$, variance $\uparrow$.
- Simple model: high bias (underfit). Complex model: high variance (overfit).
- The best model balances both, at the minimum of the test-error curve.

Diagram to draw: Bias–variance trade-off; darts board



2.5.2 Other trade-offs

- **Prediction accuracy vs. interpretability:** linear regression and trees are easy to explain; boosting and neural networks are accurate but black boxes.
- **Flexibility vs. data size:** flexible models need more data.
- **Parsimony (Occam's razor):** prefer the simplest model that explains the data.

2.6 Classification Metrics

	Predicted +	**Predicted −**
Actual +	TP	FN
Actual −	FP	TN

$$\text{Accuracy} = \frac{TP + TN}{\text{all}}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}.$$

- **Precision:** of predicted positives, how many are correct. Important when FP is costly (spam folder).
- **Recall (sensitivity):** of actual positives, how many are found. Important when FN is costly (cancer test).
- Accuracy misleads on **imbalanced** data (99% "not fraud" is 99% accurate yet useless).

Numerical pattern: confusion matrix

100 e-mails: $TP = 40$, $FN = 10$, $FP = 5$, $TN = 45$. Accuracy $= 0.85$; Precision $= 40/45 = 0.889$; Recall $= 40/50 = 0.80$; $F_1 = \frac{2(0.889)(0.8)}{1.689} = 0.842$.

2.7 Sampling Distribution of an Estimator

Definition: Estimator and sampling distribution

An **estimator** $\hat{\theta}$ is a rule (statistic) that computes an estimate of a population parameter θ from a sample (e.g. the sample mean $\bar{X}$ for μ). The **sampling distribution** of $\hat{\theta}$ is the probability distribution of its values over all possible samples of the same size.

- **Bias:** $\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta$; unbiased if 0. $\text{MSE}(\hat{\theta}) = \text{Var}(\hat{\theta}) + \text{Bias}^2$.
- **Standard error:** standard deviation of the sampling distribution. For the mean: $\text{SE}(\bar{X}) = \sigma/\sqrt{n}$.
- **Central Limit Theorem:** for large n (≥ 30), $\bar{X} \approx \mathcal{N}(\mu, \sigma^2/n)$, whatever the population shape.
- **95% confidence interval:** $\bar{x} \pm 1.96\, s/\sqrt{n}$.
- **Sample variance** $s^2 = \frac{1}{n-1}\sum(x_i - \bar{x})^2$ is unbiased; dividing by n is biased (too small).
- **Bootstrap:** estimate SE by resampling the data with replacement many times.
- 4 times more data $\Rightarrow$ SE halves.

Numerical pattern: standard error and CI

$\bar{x} = 70$, $\sigma = 10$, $n = 25$. $\text{SE} = 10/\sqrt{25} = 2$. 95% CI $= 70 \pm 1.96(2) = 70 \pm 3.92 = (66.08, 73.92)$.

Exam focus

Most asked: bias–variance trade-off (formula + diagram), training vs. test loss (U-curve), k-fold cross-validation, standard error and CLT, precision/recall numerical.

Important Questions

- **[2 marks]** What is statistical learning? Write $Y = f(X) + \varepsilon$ and explain.
- **[2 marks]** Differentiate reducible and irreducible error.
- **[2 marks]** Define risk and empirical risk.
- **[2 marks]** What is the standard error of the mean?
- **[5 marks]** Compare parametric and non-parametric methods.
- **[5 marks]** Explain k-fold cross-validation with a diagram.
- **[5 marks]** Explain training loss and test loss with a graph.
- **[5 marks]** State the central limit theorem and its importance.
- **[10 marks]** Explain the bias–variance trade-off with formula and diagram.
- **[10 marks]** Explain the sampling distribution of an estimator, bias, standard error and confidence interval with an example.

3. Distance-Based Methods, Decision Trees and Naive Bayes

Study time: 2.5 hours

3.1 Distance-Based Methods

Definition: Distance-based learning

Distance-based methods classify or predict a new instance from the training instances that are **closest** to it, using a distance (or similarity) measure. Assumption: similar inputs have similar outputs.

Measure	Formula	Note
Euclidean (L_2)	$\sqrt{\sum_i (a_i - b_i)^2}$	straight-line; most common
Manhattan (L_1)	$\sum_i \lvert a_i - b_i \rvert$	city-block distance
Minkowski (L_p)	$\left(\sum_i \lvert a_i - b_i \rvert^p\right)^{1/p}$	$p=1$: Manhattan, $p=2$: Euclidean
Chebyshev (L_∞)	$\max_i \lvert a_i - b_i \rvert$	largest single difference
Cosine similarity	$\dfrac{\boldsymbol{a}\cdot\boldsymbol{b}}{\lVert\boldsymbol{a}\rVert\lVert\boldsymbol{b}\rVert}$	text documents; direction only
Hamming	number of positions that differ	categorical / binary data

Numerical pattern: distances

$\boldsymbol{a} = (1,2)$, $\boldsymbol{b} = (4,6)$. Euclidean $= \sqrt{9+16} = 5$; Manhattan $= 3+4 = 7$; Chebyshev $= \max(3,4) = 4$.

Feature scaling is essential: a feature measured in large units (salary in rupees) dominates one in small units (age). Use

$$\text{min–max: } x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0,1], \qquad \text{z-score: } x' = \frac{x - \mu}{\sigma}.$$

3.2 k-Nearest Neighbours (k-NN)

Definition: k-Nearest Neighbours

k-NN is a supervised, **instance-based** (lazy) learning algorithm that predicts the class of a new point by the **majority vote** of its k nearest training points (classification), or by the **average** of their values (regression).

Algorithm:

1. Choose k and a distance measure; scale the features.
2. Compute the distance from the query point to every training point.
3. Sort the distances and take the k nearest neighbours.
4. Classification: majority class; regression: mean of their y values.

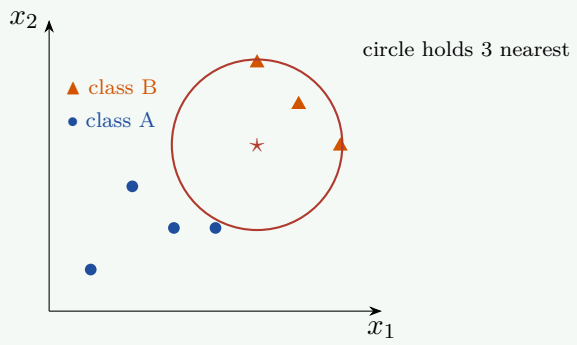
- **Lazy learner:** no training phase; all work happens at prediction time.
- **Choice of k:** small $k \Rightarrow$ noisy, overfits (high variance); large $k \Rightarrow$ too smooth, underfits (high bias). Choose k by cross-validation; use an **odd** k for two classes.
- **Weighted k-NN:** weight each neighbour by $1/d$ so closer neighbours count more.

Numerical pattern: k-NN classification

Query $(5,4)$. Training points with distance: $P_4(6,5)$ B 1.414; $P_5(7,4)$ B 2.0; $P_6(5,6)$ B 2.0; $P_7(4,2)$ A 2.236; $P_3(3,2)$ A 2.828; $P_2(2,3)$ A 3.162; $P_1(1,1)$ A 5.0.

$k = 3$: B,B,B $\Rightarrow$ **B**. $k = 5$: 3 B vs 2 A $\Rightarrow$ **B**. $k = 7$: 4 A vs 3 B $\Rightarrow$ **A** (prediction depends on k).

Diagram to draw: k-NN with $k = 3$



Advantages	simple, no training, works for multi-class, naturally non-linear boundaries
Disadvantages	slow prediction on big data, needs scaling, sensitive to irrelevant features and to k, suffers from the curse of dimensionality
Applications	recommendation, handwriting recognition, credit rating, medical diagnosis

3.3 Decision Trees

Definition: Decision tree

A **decision tree** is a supervised, tree-structured model in which each **internal node** tests a feature, each **branch** is an outcome of the test, and each **leaf node** gives a class label (or value). The tree is built top-down by choosing, at each node, the feature that best separates the classes.

Definition: Entropy, information gain and Gini index

Entropy measures the impurity of a set S: $H(S) = -\sum_i p_i \log_2 p_i$ (0 = pure, 1 = 50/50 for two classes).
Information gain is the reduction in entropy after splitting on attribute A: $\text{Gain}(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$.
Gini index: $G = 1 - \sum_i p_i^2$ (used by CART).

ID3 algorithm:

1. Compute the entropy of the whole training set.
2. For each attribute, compute the information gain.
3. Choose the attribute with the **highest gain** as the node; create a branch for each value.
4. Repeat recursively on each branch until all examples in a node have the same class (leaf) or no attributes remain.

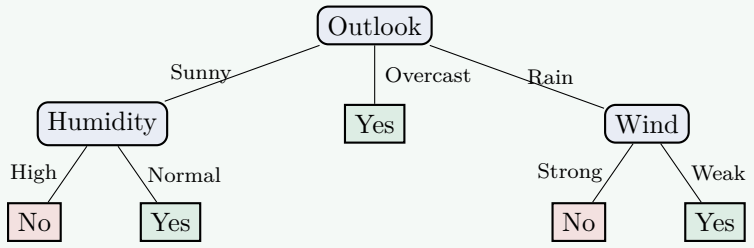
Numerical pattern: information gain (Play-Tennis, 14 days: 9 Yes, 5 No)

$H(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = \mathbf{0.940}$.
Outlook: Sunny (2Y,3N) $H = 0.971$; Overcast (4Y,0N) $H = 0$; Rain (3Y,2N) $H = 0.971$.
$\text{Gain}(\text{Outlook}) = 0.940 - [\frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971)] = 0.940 - 0.694 = \mathbf{0.246}$.
Others: Humidity 0.151, Wind 0.048, Temperature 0.029 $\Rightarrow$ **Outlook is the root**.

Numerical pattern: Gini

Node with 4 Yes, 6 No: $G = 1 - (0.4^2 + 0.6^2) = 1 - 0.52 = \mathbf{0.48}$.

Diagram to draw: Decision tree for Play-Tennis



- **ID3** uses information gain; **C4.5** uses gain ratio (handles many-valued attributes, continuous values); **CART** uses Gini and binary splits (also regression).
- **Overfitting control (pruning):** pre-pruning (max depth, min samples per leaf) or post-pruning (grow fully, then cut weak branches).

Advantages	easy to understand and draw, no scaling needed, handles numerical and categorical data, gives rules
Disadvantages	overfits easily, unstable (small data change ⇒ different tree), biased to many-valued features
Applications	loan approval, medical diagnosis, customer churn, fault diagnosis

3.4 Naive Bayes

Definition: Bayes' theorem

$$P(C \mid X) = \frac{P(X \mid C)\,P(C)}{P(X)} \qquad \text{posterior} = \frac{\text{likelihood} \times \text{prior}}{\text{evidence}}$$

Definition: Naive Bayes classifier

Naive Bayes is a probabilistic classifier based on Bayes' theorem with the **naive assumption** that features are **conditionally independent** given the class:

$$\hat{y} = \arg\max_{c} \; P(c) \prod_{j=1}^{d} P(x_j \mid c).$$

Steps: (1) compute priors $P(c)$; (2) compute likelihoods $P(x_j \mid c)$ from frequency tables; (3) multiply for each class; (4) choose the largest.

Definition: Laplace smoothing

If a feature value never occurs with a class, its probability is 0 and wipes out the whole product. **Laplace smoothing** adds α (usually 1) to every count: $P(x_j = v \mid c) = \dfrac{\text{count}(v, c) + \alpha}{N_c + \alpha V_j}$, where V_j is the number of values of feature j.

Numerical pattern: Bayes' theorem

$P(\text{spam}) = 0.3$, $P(\text{offer} \mid \text{spam}) = 0.6$, $P(\text{offer} \mid \text{ham}) = 0.1$.
$P(\text{spam} \mid \text{offer}) = \frac{0.6 \times 0.3}{0.6 \times 0.3 + 0.1 \times 0.7} = \frac{0.18}{0.25} = \mathbf{0.72}$.

Numerical pattern: smoothing

A value occurs 0 times among $N_c = 10$ examples; the feature has $V = 3$ values. Without smoothing $P = 0$; with $\alpha = 1$: $P = \frac{0+1}{10+3} = \mathbf{0.077}$.

Advantages very fast, needs little data, works well for text, handles many features and classes
Disadvantages independence assumption rarely true, zero-frequency problem, probabilities poorly calibrated
Applications spam filtering, sentiment analysis, document classification, medical diagnosis

	k-NN	**Decision tree**	**Naive Bayes**
Type	instance-based, lazy	model-based (rules)	probabilistic
Training	none	builds tree	counts frequencies
Prediction speed	slow	fast	very fast
Scaling needed	yes	no	no
Interpretability	medium	high	medium

Exam focus

Most asked: k-NN algorithm with a numerical; entropy and information gain numerical (Play-Tennis); Naive Bayes numerical with Laplace smoothing; distance measures.

Important Questions

- **[2 marks]** Define Euclidean and Manhattan distance.
- **[2 marks]** Why is k-NN called a lazy learner?
- **[2 marks]** Define entropy and information gain.
- **[2 marks]** Why is Naive Bayes called "naive"?
- **[5 marks]** Explain the effect of k in k-NN.
- **[5 marks]** Explain Bayes' theorem with an example.
- **[5 marks]** What is pruning? Explain pre- and post-pruning.
- **[10 marks]** Explain the k-NN algorithm with a numerical example; list advantages and disadvantages.
- **[10 marks]** Explain the ID3 algorithm and construct a decision tree using information gain.
- **[10 marks]** Explain the Naive Bayes classifier with Laplace smoothing and a numerical.

4. Linear Models, GLMs and SVM Basics

Study time: 2.5 hours

4.1 Linear Regression

Definition: Linear regression

Linear regression is a supervised learning method that models a numerical target as a **linear combination** of the input features plus an intercept: $\hat{y} = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d = \boldsymbol{\theta}^\top \boldsymbol{x}$. The parameters are chosen to minimise the **sum of squared errors** (least squares).

Cost function: $J(\boldsymbol{\theta}) = \frac{1}{2n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$ (MSE up to a factor).

Two ways to find $\boldsymbol{\theta}$:

- **Closed form (normal equation):** $\boldsymbol{\theta} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\boldsymbol{y}$. For one feature: $\theta_1 = S_{xy}/S_{xx}$, $\theta_0 = \bar{y} - \theta_1\bar{x}$. Exact; slow when d is large ($O(d^3)$).
- **Gradient descent:** iterative; works for huge data.

Numerical pattern: least-squares line and R^2

Hours $x = (1,2,3,4,5)$, marks $y = (35,45,50,65,70)$; $\bar{x} = 3$, $\bar{y} = 53$.
$S_{xy} = 36 + 8 + 0 + 12 + 34 = 90$, $S_{xx} = 10 \Rightarrow \theta_1 = 9$, $\theta_0 = 53 - 27 = 26$. **Model:** $\hat{y} = 26 + 9x$; $x = 6 \Rightarrow \hat{y} = 80$.
$\hat{y} = (35,44,53,62,71)$, residuals $(0,1,-3,3,-1)$, SSE $= 20$, SST $= \sum(y-\bar{y})^2 = 830$.
MSE $= 4$, RMSE $= 2$, $R^2 = 1 - \frac{20}{830} = \mathbf{0.976}$ (the line explains 97.6% of the variation).

Definition: Coefficient of determination R^2

$R^2 = 1 - \dfrac{\text{SSE}}{\text{SST}}$: the fraction of the variance of y explained by the model ($0 \le R^2 \le 1$ on training data). **Adjusted** $R^2 = 1 - (1-R^2)\frac{n-1}{n-p-1}$ penalises extra features.

4.1.1 Gradient descent

Definition: Gradient descent

Gradient descent is an iterative optimisation algorithm that minimises a cost function by repeatedly moving the parameters a small step in the direction of the **negative gradient**: $\theta_j \leftarrow \theta_j - \alpha\dfrac{\partial J}{\partial \theta_j}$, where α is the **learning rate**.

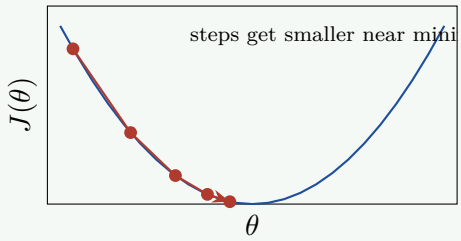
For linear regression: $\frac{\partial J}{\partial \theta_0} = \frac{1}{n}\sum(\hat{y}_i - y_i)$, $\frac{\partial J}{\partial \theta_1} = \frac{1}{n}\sum(\hat{y}_i - y_i)x_i$.

- α too small: very slow; α too large: overshoots and diverges.
- **Batch GD** uses all data per step (smooth, slow); **Stochastic GD** one example per step (fast, noisy, escapes local minima); **Mini-batch GD** small batches (best of both, used on GPUs).
- Scale features first, otherwise the cost bowl is elongated and GD is slow.

Numerical pattern: one GD step

Marks data, start $\theta_0 = \theta_1 = 0$, $\alpha = 0.01$. All $\hat{y} = 0$.
$\frac{\partial J}{\partial \theta_0} = \frac{-(35+45+50+65+70)}{5} = -53$; $\frac{\partial J}{\partial \theta_1} = \frac{-(35+90+150+260+350)}{5} = -177$.
$\theta_0 = 0 - 0.01(-53) = 0.53$, $\theta_1 = 0 - 0.01(-177) = 1.77$.

Diagram to draw: Gradient descent on a cost curve



4.1.2 Polynomial regression and regularisation

- **Polynomial regression:** add powers of features ($x^2, x^3, \dots$) and fit a linear model; high degree ⇒ overfitting.
- **Ridge (L2):** $J + \lambda \sum \theta_j^2$, shrinks weights.
- **Lasso (L1):** $J + \lambda \sum |\theta_j|$, sets some weights to zero (feature selection).
- **Elastic Net:** mix of L1 and L2. Larger $\lambda \Rightarrow$ simpler model.

Advantages	simple, fast, interpretable coefficients, closed-form solution
Disadvantages	assumes linearity, sensitive to outliers and multicollinearity
Applications	price prediction, sales forecasting, trend analysis, risk estimation

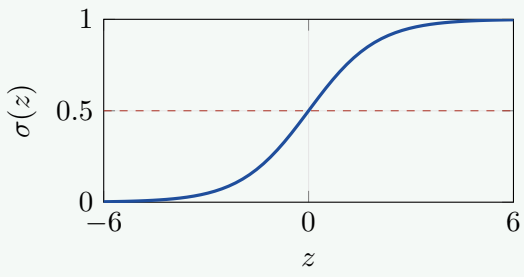
4.2 Logistic Regression

Definition: Logistic regression

Logistic regression is a supervised **classification** algorithm that estimates the probability that an instance belongs to the positive class by passing a linear combination of the features through the **sigmoid** function: $p = \sigma(\boldsymbol{\theta}^\top \boldsymbol{x}) = \dfrac{1}{1 + e^{-\boldsymbol{\theta}^\top \boldsymbol{x}}}$. Predict 1 if $p \geq 0.5$, else 0.

- Sigmoid maps any z into $(0, 1)$; $\sigma(0) = 0.5$; $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- **Odds** $= \frac{p}{1-p}$; **log-odds (logit)** $= \ln \frac{p}{1-p} = \boldsymbol{\theta}^\top \boldsymbol{x}$ (linear in x). One unit increase in x_j multiplies the odds by e^{θ_j}.
- **Decision boundary:** $\boldsymbol{\theta}^\top \boldsymbol{x} = 0$ (a straight line/plane).
- **Cost (log-loss / binary cross-entropy):** $J = -\frac{1}{n} \sum \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right]$. Convex; no closed form; minimise with GD. Gradient $\frac{1}{n} \sum (p_i - y_i) x_{ij}$.
- **Softmax regression** (multinomial): $P(y = k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$ for more than two classes.

Diagram to draw: Sigmoid curve



Numerical pattern: logistic regression

$\theta_0 = -4$, $\theta_1 = 1$, $x = 5$ hours. $z = -4 + 5 = 1$, $p = \sigma(1) = \frac{1}{1+e^{-1}} = \mathbf{0.731} \Rightarrow$ pass.
Odds $= 0.731/0.269 = 2.72 = e^1$. If $y = 1$: log-loss $= -\ln 0.731 = 0.313$.

Numerical pattern: softmax

$z = (2, 1, 0)$: $e^z = (7.389, 2.718, 1)$, sum $= 11.107 \Rightarrow P = (0.665, 0.245, 0.090)$; predict class 1.

Advantages outputs probabilities, simple, interpretable (odds ratios), fast
Disadvantages linear boundary only, needs feature engineering for complex data, sensitive to outliers
Applications credit default, disease prediction, spam detection, click-through prediction

	Linear regression	**Logistic regression**
Task	regression (number)	classification (class/probability)
Output	$(-\infty, \infty)$	$(0, 1)$
Function	identity	sigmoid
Cost	squared error (MSE)	log-loss
Solution	closed form or GD	GD only

4.3 Generalised Linear Models (GLMs)

Definition: Generalised linear model

A **GLM** extends linear regression to targets that are not normally distributed. It has three parts: (1) a **random component**: y follows an exponential-family distribution (normal, Bernoulli, Poisson, ...); (2) a **linear predictor** $\eta = \boldsymbol{\theta}^\top \boldsymbol{x}$; (3) a **link function** g with $g(\mu) = \eta$, where $\mu = \mathbb{E}[y]$.

Model	**Distribution of y**	**Link $g(\mu)$**	**Example**
Linear regression	Normal	identity μ	price
Logistic regression	Bernoulli	logit $\ln \frac{\mu}{1-\mu}$	pass/fail
Poisson regression	Poisson	log $\ln \mu$	number of calls per hour

Numerical pattern: Poisson regression

$\ln \mu = 0.5 + 0.3x$; $x = 2$: $\eta = 1.1$, $\mu = e^{1.1} = 3.0$ calls. $P(y = 0) = e^{-3} = 0.0498$. Each extra unit of x multiplies the rate by $e^{0.3} = 1.35$.

GLMs are fitted by **maximum likelihood**.

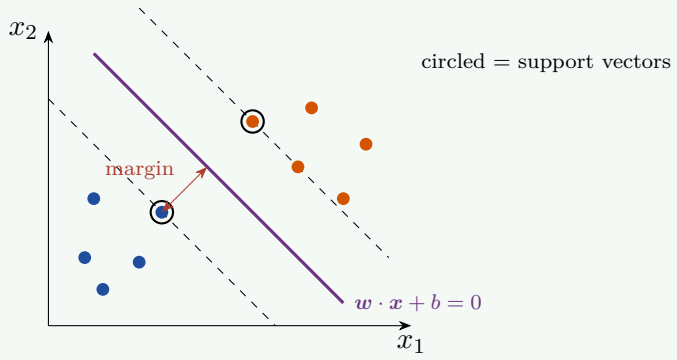
4.4 Support Vector Machines (basics)

Definition: Support vector machine

A **support vector machine (SVM)** is a supervised classifier that finds the **hyperplane** $\boldsymbol{w} \cdot \boldsymbol{x} + b = 0$ separating the two classes with the **maximum margin**. The training points closest to the hyperplane, which define the margin, are the **support vectors**.

- Labels $y_i \in \{-1, +1\}$. Margin width $= \frac{2}{\|\boldsymbol{w}\|}$.
- **Hard margin:** $\min \frac{1}{2}\|\boldsymbol{w}\|^2$ s.t. $y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1$ (only for separable data; sensitive to outliers).
- **Soft margin:** allow slack ξ_i: $\min \frac{1}{2}\|\boldsymbol{w}\|^2 + C \sum \xi_i$. Large C = few violations, narrow margin; small C = wide margin, more violations.
- **Hinge loss:** $\max(0,\ 1 - y\, f(\boldsymbol{x}))$.

Diagram to draw: Maximum-margin hyperplane



Numerical pattern: margin

Points $(1,1)$ with $y=-1$ and $(3,3)$ with $y=+1$. Solution $\boldsymbol{w}=(0.5,0.5)$, $b=-2$.
Check: $(3,3) \to 1.5+1.5-2=+1$; $(1,1) \to 0.5+0.5-2=-1$ (both on the margin). Margin $= \frac{2}{\|\boldsymbol{w}\|} = \frac{2}{0.707} = \mathbf{2.83}$ = distance between the points. New point $(4,1)$: $2.5-2=0.5>0 \Rightarrow +1$.

4.5 Binary Classification

- **Binary classifier:** two classes (positive/negative), e.g. "digit 5 or not".
- Classifier outputs a **score**; a **threshold** turns it into a class. Raising the threshold $\Rightarrow$ precision $\uparrow$, recall $\downarrow$ (**precision–recall trade-off**).
- **ROC curve:** true positive rate (recall) vs. false positive rate ($\frac{FP}{FP+TN}$) for all thresholds. **AUC** $= 1$ perfect, 0.5 random.
- **Multi-class from binary:** One-vs-Rest (K classifiers) or One-vs-One ($K(K-1)/2$ classifiers).

Exam focus

Most asked: least-squares line + R^2 numerical; gradient descent; logistic regression (sigmoid, odds, log-loss); linear vs. logistic table; GLM components; SVM margin and support vectors.

Important Questions

- **[2 marks]** Define the learning rate. What if it is too large?
- **[2 marks]** What is the sigmoid function? Write its range.
- **[2 marks]** What are support vectors?
- **[2 marks]** What is a link function?
- **[5 marks]** Compare linear and logistic regression.
- **[5 marks]** Explain batch, stochastic and mini-batch gradient descent.
- **[5 marks]** Explain Ridge and Lasso regularisation.
- **[5 marks]** Explain the ROC curve and the precision–recall trade-off.
- **[10 marks]** Fit a least-squares line to given data, predict a value and compute R^2.
- **[10 marks]** Explain logistic regression with its cost function and decision boundary.
- **[10 marks]** Explain generalised linear models with examples.

5. Ensemble Learning

Study time: 2.5 hours

Definition: Ensemble learning

Ensemble learning combines the predictions of several models (**base learners**) to obtain a prediction that is more accurate and more stable than that of any single model. A group of predictors is an **ensemble**; the idea is the "**wisdom of the crowd**".

- Works best when base learners are **diverse** (make different, independent errors) and each is better than random guessing (a **weak learner**).
- Diversity comes from: different algorithms, different training subsets, different feature subsets.
- Main methods: **voting**, **bagging/pasting**, **random forests**, **boosting**, **stacking**.

5.1 Voting Classifiers

Definition: Voting classifier

A **voting classifier** trains several different classifiers on the same data and predicts the class that gets the most votes (**hard voting**) or the class with the highest **average predicted probability** (**soft voting**).

Numerical pattern: majority vote of 3 independent classifiers, each 70% accurate

$P(\text{majority correct}) = p^3 + 3p^2(1-p) = 0.343 + 3(0.49)(0.3) = 0.343 + 0.441 = \mathbf{0.784}$ (better than 0.70).

Numerical pattern: hard vs. soft voting

$P(A)$ from three models $= 0.9,\ 0.4,\ 0.4$. Hard vote: A, B, B $\Rightarrow$ **B**. Soft vote: mean $P(A) = \frac{1.7}{3} = 0.567 > 0.5 \Rightarrow$ **A**. Soft voting uses confidence and is usually better.

5.2 Bagging and Pasting

Definition: Bagging and pasting

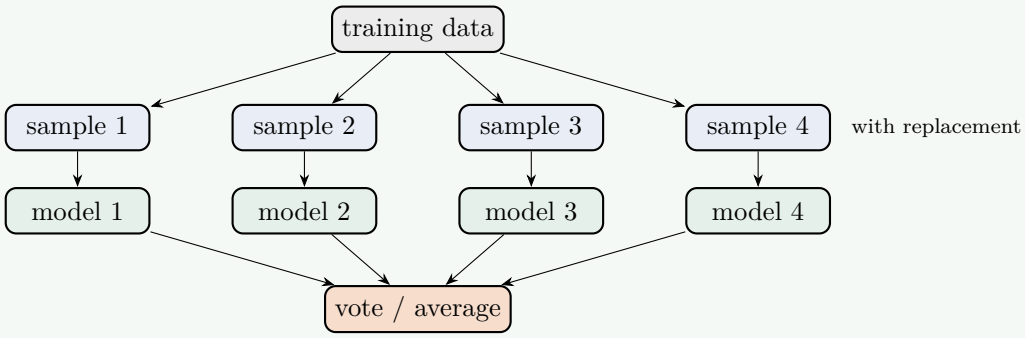
Bagging (Bootstrap Aggregating) trains the same algorithm on different random subsets of the training set drawn **with replacement** (bootstrap samples) and aggregates their predictions (vote for classification, average for regression). **Pasting** is the same but samples **without replacement**.

Algorithm:

1. For $b = 1, \ldots, B$: draw a bootstrap sample of size n from the training data.
2. Train a model (often a deep decision tree) on it.
3. Predict by majority vote (classification) or average (regression).

- Reduces **variance** without increasing bias; models train **in parallel**.
- Variance of an average of B models with correlation ρ: $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. (Example: $\rho = 0.5, \sigma^2 = 1, B = 10$: 0.55 instead of 1.)
- **Out-of-bag (OOB) evaluation:** each bootstrap sample leaves out about $(1 - \frac{1}{n})^n \approx e^{-1} = \mathbf{36.8}\%$ of the instances; they act as a free validation set.

Diagram to draw: Bagging



5.3 Random Forests

Definition: Random forest

A **random forest** is an ensemble of decision trees trained by bagging, in which each split considers only a **random subset of features** (typically $\sqrt{d}$ for classification). The final prediction is the majority vote (or average) of all trees.

- Two sources of randomness: bootstrap rows + random features ⇒ less correlated trees ⇒ lower variance than plain bagging.
- **Extra-Trees (extremely randomised trees):** also random split thresholds; faster.
- **Feature importance:** average decrease in impurity caused by each feature over all trees.
- Hyperparameters: number of trees, max features, max depth, min samples per leaf.

Advantages	high accuracy, resists overfitting, handles many features, gives feature importance and OOB score, little tuning
Disadvantages	less interpretable than one tree, slow and large with many trees
Applications	fraud detection, credit scoring, disease prediction, remote sensing, stock analysis

5.4 Boosting

Definition: Boosting

Boosting trains weak learners **sequentially**, each new learner focusing on the mistakes of the previous ones, and combines them into a strong learner by a weighted vote.

5.4.1 AdaBoost (Adaptive Boosting)

Algorithm:

1. Give every training instance equal weight $w_i = \frac{1}{n}$.
2. Train a weak learner h_t (e.g. a decision stump); compute weighted error $\varepsilon_t = \sum_{\text{wrong}} w_i$.
3. Learner weight $\alpha_t = \frac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}$ (more accurate ⇒ larger say).
4. Update $w_i \leftarrow w_i \, e^{-\alpha_t y_i h_t(\boldsymbol{x}_i)}$: wrong instances get **heavier**; normalise to sum 1.
5. Repeat; final $H(\boldsymbol{x}) = \text{sign}\left(\sum_t \alpha_t h_t(\boldsymbol{x})\right)$.

Numerical pattern: one AdaBoost round, 5 instances

$w = (0.2, \ldots, 0.2)$; h_1 wrong only on instance 4: $\varepsilon_1 = 0.2$, $\alpha_1 = \frac{1}{2} \ln \frac{0.8}{0.2} = \frac{1}{2} \ln 4 = \mathbf{0.693}$.
Correct $\times e^{-0.693} = \times 0.5 \to 0.1$; wrong $\times e^{0.693} = \times 2 \to 0.4$. Sum $= 0.8$; normalised $w = (0.125, 0.125, 0.125, \mathbf{0.5}, 0.125)$.
Instance 4 now carries half the weight, so the next learner must get it right.

5.4.2 Gradient boosting

- Each new tree is fitted to the **residual errors** $r_i = y_i - F_{m-1}(\boldsymbol{x}_i)$ of the current ensemble; $F_m = F_{m-1} + \eta\, h_m$.
- η (learning rate, **shrinkage**) small $\Rightarrow$ more trees needed but better generalisation. Use early stopping.
- **XGBoost, LightGBM, CatBoost**: fast, regularised implementations.

Numerical pattern: gradient boosting

$y = (10, 20, 30)$. $F_0 = \bar{y} = 20$; residuals $(-10, 0, 10)$. A tree fits them exactly; with $\eta = 0.1$: $F_1 = 20 + 0.1(-10, 0, 10) = (19, 20, 21)$. New residuals $(-9, 0, 9)$: error shrinks step by step.

	Bagging	**Boosting**
Training	parallel, independent	sequential, each fixes the last
Sampling	bootstrap samples	reweighted (or residual) data
Weights of models	equal	by accuracy (α_t)
Reduces	variance	bias (and variance)
Base learner	strong, deep trees	weak learners (stumps)
Overfitting	rarely	possible if too many rounds
Outliers / noise	robust	sensitive
Example	random forest	AdaBoost, gradient boosting

Advantages very high accuracy, reduces bias, works with simple learners
Disadvantages sequential (slow to train), sensitive to noise and outliers, needs careful tuning
Applications web search ranking, face detection (AdaBoost), Kaggle competitions, credit risk

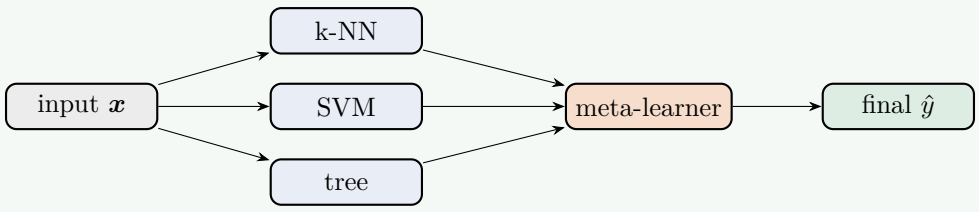
5.5 Stacking

Definition: Stacking (stacked generalisation)

Stacking trains several different base models (level 0) and then trains a **meta-learner** (blender, level 1) that takes the base models' predictions as its inputs and learns how best to combine them.

Steps: (1) split training data into two parts (or use cross-validation predictions); (2) train base models on part 1; (3) predict part 2 with them; (4) train the meta-learner on these predictions with the true labels; (5) for new data: base predictions $\rightarrow$ meta-learner $\rightarrow$ final output.

Diagram to draw: Stacking



Method	**Base models**	**Data per model**	**Combination**	**Training**
Voting	different	same	majority/average	parallel
Bagging	same	bootstrap samples	majority/average	parallel
Boosting	same (weak)	reweighted	weighted vote	sequential
Stacking	different	same (CV)	learned meta-model	two levels

Exam focus

Most asked: bagging vs. boosting table; random forest; AdaBoost algorithm with weights; stacking diagram; hard vs. soft voting.

Important Questions

- **[2 marks]** Define ensemble learning.
- **[2 marks]** Differentiate bagging and pasting.
- **[2 marks]** What is out-of-bag evaluation?
- **[2 marks]** What is a weak learner?
- **[5 marks]** Explain hard and soft voting with an example.
- **[5 marks]** Explain random forests. Why are they better than a single tree?
- **[5 marks]** Explain stacking with a diagram.
- **[10 marks]** Compare bagging and boosting. Explain the AdaBoost algorithm.
- **[10 marks]** Explain ensemble learning methods: voting, bagging, random forest, boosting and stacking.

6. Support Vector Machines and Naive Bayes Classifiers

Study time: 2 hours

6.1 Linear SVM Classification

Definition: Large-margin classification

A **linear SVM** separates two classes with the hyperplane $\boldsymbol{w} \cdot \boldsymbol{x} + b = 0$ that has the **widest possible margin** ("widest street") between the classes. Only the **support vectors** (instances on the edge of the street) determine it; adding points outside the street does not change it.

- Decision: $\hat{y} = +1$ if $\boldsymbol{w} \cdot \boldsymbol{x} + b \geq 0$, else -1. Margin $= 2/\|\boldsymbol{w}\|$; maximising it $\equiv$ minimising $\frac{1}{2}\|\boldsymbol{w}\|^2$.
- **Hard-margin:** all instances off the street; works only if data is linearly separable; very sensitive to outliers.
- **Soft-margin:** balance wide street vs. margin violations using slack $\xi_i \geq 0$:

$$\min \ \tfrac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_i \xi_i \quad \text{s.t.} \quad y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1 - \xi_i.$$

- **Hyperparameter C:** large $C \Rightarrow$ narrow margin, few violations (overfit risk); small $C \Rightarrow$ wide margin, more violations (better generalisation). If the SVM overfits, **reduce** C.
- SVMs are sensitive to **feature scaling**: always standardise.
- **Dual form:** $f(\boldsymbol{x}) = \sum_i \alpha_i y_i (\boldsymbol{x}_i \cdot \boldsymbol{x}) + b$, with $0 \leq \alpha_i \leq C$; $\alpha_i > 0$ only for support vectors. The data appears only through dot products $\Rightarrow$ kernel trick.

6.2 Non-linear SVM Classification

- **Idea:** map data to a higher-dimensional space $\varphi(\boldsymbol{x})$ where it becomes linearly separable. Example: 1-D points x not separable $\rightarrow$ add x^2 $\rightarrow$ separable in 2-D.
- Adding polynomial features explicitly is slow for high degrees.

Definition: Kernel trick

A **kernel** is a function $K(\boldsymbol{a}, \boldsymbol{b}) = \varphi(\boldsymbol{a}) \cdot \varphi(\boldsymbol{b})$ that computes the dot product in the transformed space **without computing the transformation**. The **kernel trick** replaces every dot product in the SVM by K, giving a non-linear classifier at the cost of a linear one.

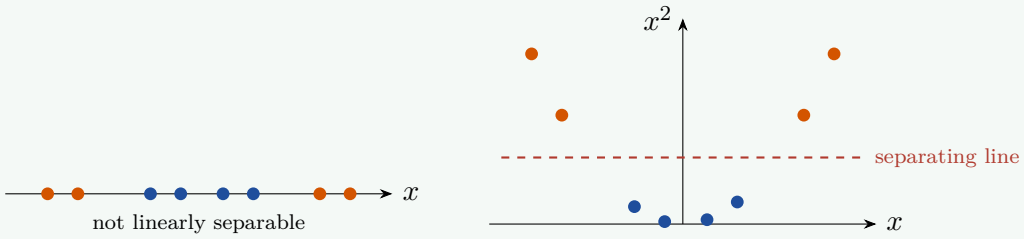
Kernel	Formula	Hyperparameters
Linear	$\boldsymbol{a} \cdot \boldsymbol{b}$	C
Polynomial	$(\gamma\, \boldsymbol{a} \cdot \boldsymbol{b} + r)^d$	degree d, r, C
Gaussian RBF	$\exp(-\gamma\|\boldsymbol{a} - \boldsymbol{b}\|^2)$	γ, C
Sigmoid	$\tanh(\gamma\, \boldsymbol{a} \cdot \boldsymbol{b} + r)$	γ, r

- **RBF γ:** large $\gamma \Rightarrow$ narrow bell, wiggly boundary (overfit); small $\gamma \Rightarrow$ smooth boundary (underfit). If overfitting, reduce γ (and C).
- Try linear first (large data, text); then RBF.

Numerical pattern: kernels

Polynomial $K = (\boldsymbol{a} \cdot \boldsymbol{b})^2$, $\boldsymbol{a} = (1, 2)$, $\boldsymbol{b} = (3, 4)$: $\boldsymbol{a} \cdot \boldsymbol{b} = 11$, $K = \mathbf{121}$.
Check with $\varphi(\boldsymbol{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$: $\varphi(\boldsymbol{a}) = (1, 2.83, 4)$, $\varphi(\boldsymbol{b}) = (9, 16.97, 16)$, dot $= 9+48+64 = 121$ ✓.
RBF $\gamma = 0.1$, $\boldsymbol{a} = (1, 2)$, $\boldsymbol{b} = (2, 4)$: $\|\boldsymbol{a} - \boldsymbol{b}\|^2 = 5$, $K = e^{-0.5} = \mathbf{0.607}$ (similarity).

Diagram to draw: Lifting 1-D data with x^2



6.3 SVM Regression (SVR)

Definition: Support vector regression

SVR reverses the SVM objective: it fits as many instances as possible **inside** a tube (street) of half-width ε around the regression function, while keeping the function as flat as possible. Errors inside the tube are ignored (ε-**insensitive**); only points outside the tube are penalised.

- ε-insensitive loss: $L_\varepsilon = \max\left(0,\ |y - f(\boldsymbol{x})| - \varepsilon\right)$.
- Problem: $\min\ \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum(\xi_i + \xi_i^*)$. Support vectors lie on or outside the tube.
- Larger $\varepsilon \Rightarrow$ fewer support vectors, flatter model. Kernels (polynomial, RBF) give non-linear SVR.

Numerical pattern: ε-insensitive loss, $\varepsilon = 0.5$, $y = 10$

$f = 10.3$: $|-0.3| < 0.5 \Rightarrow L = 0$. $f = 11$: $L = 1 - 0.5 = \mathbf{0.5}$. $f = 9$: $L = 0.5$.

Advantages	effective in high dimensions, memory efficient (uses only support vectors), flexible with kernels, robust (max margin)
Disadvantages	slow on very large data ($O(n^2)$–$O(n^3)$), needs scaling and tuning of C, γ, no direct probabilities, hard to interpret
Applications	text and image classification, face detection, bioinformatics, handwriting recognition, stock prediction (SVR)

6.4 Naive Bayes Classifiers

All use $\hat{y} = \arg\max_c \left[\ln P(c) + \sum_j \ln P(x_j \mid c)\right]$ (logs avoid underflow); they differ in how $P(x_j \mid c)$ is modelled.

Definition: Gaussian Naive Bayes

For **continuous** features, assumes each feature in each class is normally distributed: $P(x_j \mid c) = \frac{1}{\sqrt{2\pi}\sigma_{jc}} \exp\Big(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\Big)$, with mean and variance estimated per class. Example: classify fruit by weight and sweetness.

Definition: Multinomial Naive Bayes

For **count** features (word frequencies in text): $P(w \mid c) = \frac{N_{wc} + \alpha}{N_c + \alpha V}$, where N_{wc} = count of word w in class c, N_c = total words in class c, V = vocabulary size.

Definition: Bernoulli Naive Bayes

For **binary** features (word present or absent): $P(x_j \mid c) = p_{jc}^{x_j}(1 - p_{jc})^{1-x_j}$. Unlike multinomial, it also penalises **absent** words. Good for short texts.

Numerical pattern: Multinomial NB

Spam: "free win", "free offer". Ham: "meeting today", "free meeting". $V = 5$, $N_{\text{spam}} = 4$, $N_{\text{ham}} = 4$, priors 0.5. Test: "free win" ($\alpha = 1$).
Spam: $P(\text{free}) = \frac{2+1}{9}$, $P(\text{win}) = \frac{1+1}{9} \Rightarrow 0.5 \times \frac{3}{9} \times \frac{2}{9} = 0.0370$.
Ham: $P(\text{free}) = \frac{1+1}{9}$, $P(\text{win}) = \frac{0+1}{9} \Rightarrow 0.5 \times \frac{2}{9} \times \frac{1}{9} = 0.0123$.
Spam ($P = \frac{0.0370}{0.0493} = 0.75$).

	Gaussian NB	**Multinomial NB**	**Bernoulli NB**
Feature type	continuous	counts	binary (0/1)
Likelihood	normal density	word frequency	presence/absence
Absent features	—	ignored	penalised
Use	sensor, medical data	document classification	short text, spam

Advantages fast to train and predict, works with small data and many features, good baseline for text

Disadvantages independence assumption, zero-frequency problem (needs smoothing), poor probability estimates

Applications spam filtering, sentiment analysis, news categorisation, real-time prediction

	SVM	**Naive Bayes**
Type	discriminative (boundary)	generative (probabilistic)
Speed	slow on large data	very fast
Data needed	medium	small
Output	class (score)	class probabilities

Exam focus

Most asked: soft vs. hard margin and C; kernel trick with polynomial/RBF kernels; effect of γ; SVR and the ε-tube; three types of Naive Bayes with a table.

Important Questions

- **[2 marks]** What is the kernel trick?
- **[2 marks]** Write the RBF kernel and explain γ.
- **[2 marks]** What is the ε-insensitive loss?
- **[2 marks]** Differentiate multinomial and Bernoulli Naive Bayes.
- **[5 marks]** Explain hard-margin and soft-margin classification.
- **[5 marks]** Explain the role of hyperparameters C and γ in SVM.
- **[5 marks]** Explain SVM regression with a diagram.
- **[10 marks]** Explain linear and non-linear SVM classification with the kernel trick.
- **[10 marks]** Explain Gaussian, multinomial and Bernoulli Naive Bayes with an example.

7. Clustering

Study time: 2.5 hours

Definition: Clustering

Clustering is an **unsupervised** learning task that groups unlabelled instances into **clusters** so that instances in the same cluster are similar to each other and dissimilar to instances in other clusters.

Applications: customer segmentation, data analysis, dimensionality reduction, anomaly (outlier) detection, semi-supervised learning, search engines (similar images), image segmentation.

Types: centroid-based (K-Means), density-based (DBSCAN), distribution-based (Gaussian mixtures), hierarchical (agglomerative).

7.1 K-Means

Definition: K-Means

K-Means partitions n instances into k clusters, each represented by its **centroid** (mean), by minimising the **inertia** (within-cluster sum of squared distances): $J = \sum_{j=1}^{k} \sum_{\boldsymbol{x}_i \in C_j} \|\boldsymbol{x}_i - \boldsymbol{\mu}_j\|^2$.

Algorithm (Lloyd's):

1. Choose k; initialise k centroids (e.g. k random instances).
2. **Assign** each instance to its nearest centroid.
3. **Update** each centroid to the mean of its assigned instances.
4. Repeat 2–3 until the centroids stop moving.

Inertia never increases, so K-Means always converges, but possibly to a **local** optimum.

Numerical pattern: K-Means, $k = 2$

Points $A(2,3)$, $B(5,3)$, $C(5,2)$, $D(1,1)$, $E(4,4)$, $F(4,2)$. Start $\boldsymbol{\mu}_1 = C(5,2)$, $\boldsymbol{\mu}_2 = F(4,2)$.
Iter 1: distances to $(5,2)$ and $(4,2)$: A 3.16/2.24 $\to$ 2; B 1.0/1.41 $\to$ 1; C 0/1 $\to$ 1; D 4.12/3.16 $\to$ 2; E 2.24/2.0 $\to$ 2; F 1/0 $\to$ 2.
Clusters $\{B, C\}$, $\{A, D, E, F\}$. New $\boldsymbol{\mu}_1 = (5, 2.5)$, $\boldsymbol{\mu}_2 = (2.75, 2.5)$.
Iter 2: E (1.80 vs 1.95) and F (1.12 vs 1.35) move to cluster 1. Clusters $\{B, C, E, F\}$, $\{A, D\}$; $\boldsymbol{\mu}_1 = (4.5, 2.75)$, $\boldsymbol{\mu}_2 = (1.5, 2)$.
Iter 3: no change $\Rightarrow$ **converged**. Inertia $= 3.75 + 2.5 = 6.25$.

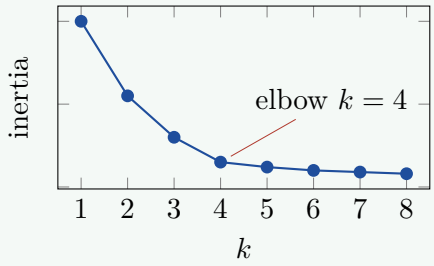
7.1.1 Choosing k and better initialisation

- **K-Means++:** choose the first centroid at random, then each next one with probability $\propto D(\boldsymbol{x})^2$ (distance to nearest chosen centroid) $\Rightarrow$ well spread starting centroids.
- **Multiple runs:** run with different random starts (`n_init`) and keep the lowest inertia.
- **Mini-batch K-Means:** uses small batches; much faster on big data.
- **Elbow method:** plot inertia vs. k; choose the k at the "elbow" where the decrease slows.
- **Silhouette coefficient:** $s(i) = \frac{b-a}{\max(a,b)} \in [-1, 1]$, a = mean distance to own cluster, b = mean distance to nearest other cluster. $+1$: well placed; 0: on boundary; -1: wrong cluster. Choose k with highest mean silhouette.

Numerical pattern: silhouette

$a = 2$, $b = 5$: $s = \frac{5-2}{5} = \mathbf{0.6}$ (well clustered).

Diagram to draw: Elbow curve



7.1.2 Limits of K-Means

- k must be specified in advance; result depends on initialisation.
- Assumes **spherical, equal-size, equal-density** clusters; fails on elongated or non-convex shapes (e.g. moons, rings).
- Sensitive to **outliers** (the mean is pulled) and to **feature scale**.
- Hard assignment only (no probabilities).

Advantages	simple, fast, scales to large data, easy to interpret
Disadvantages	needs k, local optima, only spherical clusters, sensitive to outliers and scaling
Applications	customer segmentation, image compression/segmentation, document clustering, vector quantisation

7.2 Uses of Clustering

- **Image segmentation:** partition an image into segments. **Colour segmentation** clusters the pixel colours (RGB vectors) with K-Means and replaces each pixel by its cluster's mean colour. Example: with $k = 8$ colours each pixel needs 3 bits instead of 24 $\Rightarrow$ about 8× smaller.
- **Preprocessing:** replace each instance by its distances to the k centroids (a new, often smaller, feature set) before a classifier. Example: digits data, K-Means with 50 clusters then logistic regression improves accuracy; k tuned with grid search.
- **Semi-supervised learning:** cluster all data and label only the instance nearest each centroid, called a **representative instance**; then **propagate** the label to all instances of the cluster. Few labels give high accuracy.
- **Active learning:** the model asks a human to label the instances it is least sure about.

7.3 DBSCAN

Definition: DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) defines clusters as continuous regions of **high density**. With parameters ε (radius) and **MinPts**, every instance is a **core point** (at least MinPts instances, including itself, within distance ε), a **border point** (not core but within ε of a core point), or **noise** (neither).

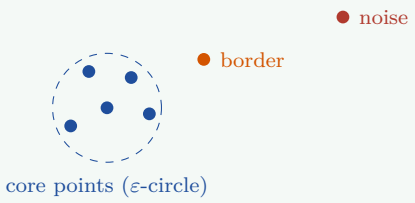
Algorithm:

1. For each instance count its ε-neighbours; mark core points.
2. Connect core points that are within ε of each other; each connected group is a cluster.
3. Assign border points to a neighbouring core point's cluster.
4. Remaining points are noise (anomalies).

Numerical pattern: DBSCAN, $\varepsilon = 1.5$, MinPts $= 3$

$A(1,1)$, $B(1,2)$, $C(2,1)$, $D(2,2)$, $E(3,3)$, $F(8,8)$.
A: neighbours B, C, D (≤ 1.41) $\Rightarrow 4 \geq 3$: **core**. Similarly B, C, D are core (D also reaches E at 1.41).
E: only D within 1.5 $\Rightarrow 2 < 3$, but near core D: **border**. F: alone: **noise**.
Result: one cluster $\{A, B, C, D, E\}$, noise $\{F\}$.

Diagram to draw: Core, border and noise points



Advantages no need to specify k, finds clusters of any shape, detects outliers

Disadvantages hard to choose ε and MinPts, fails when densities vary, slow in high dimensions, cannot predict new points directly

Applications anomaly detection, geographic (GPS) data, fraud detection, astronomy

	K-Means	DBSCAN
Basis	distance to centroids	density
Number of clusters	must give k	found automatically
Cluster shape	spherical	arbitrary
Outliers	pulled into clusters	labelled as noise
Parameters	k	ε, MinPts

7.4 Gaussian Mixture Models (GMM)

Definition: Gaussian mixture model

A **GMM** is a probabilistic model that assumes the data are generated from a mixture of k Gaussian distributions with unknown parameters: $p(\boldsymbol{x}) = \sum_{j=1}^{k} \pi_j \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$, $\sum \pi_j = 1$. Each cluster is an ellipse with its own mean, covariance (shape/size/orientation) and weight π_j.

Expectation–Maximisation (EM) algorithm:

1. Initialise $\pi_j, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j$ (e.g. from K-Means).
2. **E-step:** compute the **responsibility** of each cluster for each instance: $\gamma_{ij} = \dfrac{\pi_j \mathcal{N}(\boldsymbol{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}{\sum_l \pi_l \mathcal{N}(\boldsymbol{x}_i \mid \boldsymbol{\mu}_l, \boldsymbol{\Sigma}_l)}$.
3. **M-step:** update parameters using the responsibilities as weights: $N_j = \sum_i \gamma_{ij}$, $\boldsymbol{\mu}_j = \frac{1}{N_j} \sum_i \gamma_{ij} \boldsymbol{x}_i$, $\pi_j = N_j / n$, $\boldsymbol{\Sigma}_j =$ weighted covariance.
4. Repeat until the log-likelihood stops increasing.

Numerical pattern: E-step in 1-D

$\pi_1 = \pi_2 = 0.5$, $\mu_1 = 0$, $\mu_2 = 4$, $\sigma = 1$, point $x = 1$.
$\mathcal{N}(1 \mid 0, 1) = 0.3989 e^{-0.5} = 0.2420$; $\mathcal{N}(1 \mid 4, 1) = 0.3989 e^{-4.5} = 0.0044$.
$\gamma_1 = \frac{0.2420}{0.2420 + 0.0044} = \mathbf{0.982}$: the point belongs 98% to cluster 1 (**soft clustering**).

- EM is like K-Means with **soft** assignments and elliptical clusters.
- **Choosing k:** minimise **BIC** $= p \ln n - 2 \ln \hat{L}$ or **AIC** $= 2p - 2 \ln \hat{L}$.
- **Anomaly detection:** instances in low-density regions (below a density threshold) are anomalies.

- GMM is a **generative** model: it can sample new instances.

Advantages	soft (probabilistic) clustering, elliptical clusters of different sizes, density estimation and anomaly detection, generative
Disadvantages	needs k, local optima, slow, many parameters, assumes Gaussian shape
Applications	speaker identification, anomaly detection, image segmentation, finance

Exam focus

Most asked: K-Means algorithm with numerical; limits of K-Means; elbow and silhouette; DBSCAN core/border/noise; K-Means vs. DBSCAN; GMM with EM; image segmentation and semi-supervised learning using clustering.

Important Questions

- [**2 marks**] Define clustering. Give two applications.
- [**2 marks**] What is inertia?
- [**2 marks**] Define core point, border point and noise in DBSCAN.
- [**2 marks**] What is a responsibility in GMM?
- [**5 marks**] Explain the elbow method and silhouette score.
- [**5 marks**] Explain the limits of K-Means.
- [**5 marks**] How is clustering used for image segmentation and preprocessing?
- [**5 marks**] Explain semi-supervised learning using clustering.
- [**10 marks**] Explain K-Means with an example and its limitations.
- [**10 marks**] Explain DBSCAN and compare it with K-Means.
- [**10 marks**] Explain Gaussian mixture models and the EM algorithm.

8. Dimensionality Reduction

Study time: 2 hours

Definition: Dimensionality reduction

Dimensionality reduction is the process of reducing the number of features (dimensions) of a dataset while keeping as much of the important information as possible.

Why: speed up training, remove noise and redundant features, avoid the curse of dimensionality, compress data, visualise data in 2-D or 3-D (**data visualisation**). **Cost:** some information is lost; pipelines become more complex.

8.1 The Curse of Dimensionality

Definition: Curse of dimensionality

The **curse of dimensionality** refers to the problems that arise when data has very many features: the space becomes so large that data is **sparse**, distances between points become large and similar, and the amount of data needed to generalise grows **exponentially** with the number of dimensions.

- In a unit square, a random point is near the border (within 0.001) with probability 0.4%; in 10 000 dimensions $> 99.999999\%$. In general $P = 1 - (1 - 2\delta)^d$.
- Average distance between two random points: 0.52 in 2-D, 0.66 in 3-D, about 408.25 in 1 000 000-D $\Rightarrow$ new instances are far from training instances.
- To cover a space with b bins per feature you need b^d cells (e.g. 10^{100} for 100 features).
- Effect: high risk of **overfitting**; distance-based methods (k-NN, K-Means) suffer most.
- Remedy: more data (impractical: exponential) or **reduce dimensions**.

Numerical pattern: near the border

$\delta = 0.001$. $d = 2$: $1 - 0.998^2 = 0.004$ (0.4%). $d = 1000$: $1 - 0.998^{1000} = 1 - e^{-2} = \mathbf{0.865}$ (86%).

8.2 Main Approaches

Definition: Projection

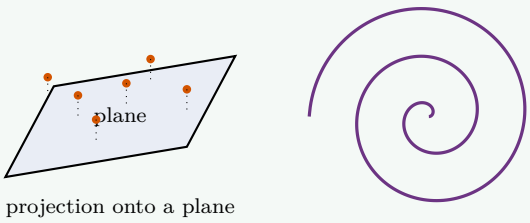
In most real problems the training instances lie close to a much **lower-dimensional subspace**. **Projection** drops perpendicular onto that subspace (e.g. 3-D points near a plane are projected to 2-D). PCA is a projection method.

Definition: Manifold learning

A d-dimensional **manifold** is a shape that locally looks like a d-dimensional plane but is bent or twisted in a higher-dimensional space (e.g. the **Swiss roll**: 2-D manifold in 3-D). **Manifold learning** models this shape and **unrolls** it; projection would squash its layers together. Examples: LLE, Isomap, t-SNE.

- **Manifold hypothesis:** most real high-dimensional data lie close to a much lower-dimensional manifold.
- **Feature selection** keeps a subset of original features; **feature extraction** creates new combined features (PCA).
- Linear methods: PCA, LDA. Non-linear: Kernel PCA, LLE, t-SNE, autoencoders.

Diagram to draw: Projection vs. manifold (Swiss roll)



projection onto a plane

Swiss roll: unroll, do not squash

8.3 Principal Component Analysis (PCA)

Definition: PCA

Principal Component Analysis is an unsupervised, linear dimensionality-reduction technique that finds new orthogonal axes, the **principal components**, ordered so that the first captures the **maximum variance** of the data, the second the maximum remaining variance, and so on. The data is then projected onto the first m components.

- **Preserving variance:** the axis that keeps maximum variance also minimises the mean squared distance between original points and their projections (least information lost).
- Components are the **eigenvectors** of the covariance matrix; their **eigenvalues** are the variances along them.
- PCA assumes data is **centred**; scale features first if units differ.

Algorithm:

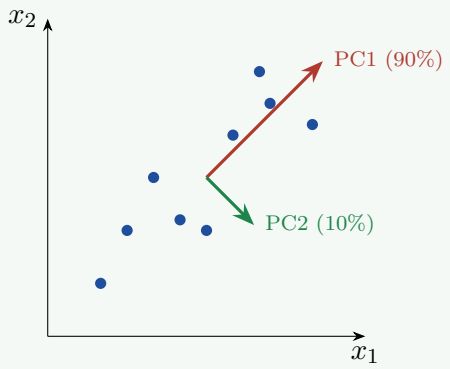
1. Standardise / centre the data: $\tilde{\mathbf{X}} = \mathbf{X} - \bar{\mathbf{x}}$.
2. Compute the covariance matrix $\mathbf{C} = \frac{1}{n-1}\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$.
3. Find eigenvalues and eigenvectors: $\mathbf{C}\boldsymbol{w} = \lambda\boldsymbol{w}$; sort by λ (descending).
4. Choose m components (e.g. 95% explained variance).
5. Project: $\mathbf{Z} = \tilde{\mathbf{X}}\mathbf{W}_m$.

In practice PCA is computed with **SVD**: $\tilde{\mathbf{X}} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$; rows of $\mathbf{V}^\top$ are the components, $\lambda_j = s_j^2/(n-1)$.

Numerical pattern: PCA by hand

Points $(1,1),(2,3),(3,2),(4,5),(5,4)$. Means $\bar{x} = \bar{y} = 3$; centred: $(-2,-2),(-1,0),(0,-1),(1,2),(2,1)$.
$s_{xx} = \frac{10}{4} = 2.5$, $s_{yy} = 2.5$, $s_{xy} = \frac{4+0+0+2+2}{4} = 2 \Rightarrow \mathbf{C} = \left[\begin{smallmatrix} 2.5 & 2 \\ 2 & 2.5 \end{smallmatrix}\right]$.
$\det(\mathbf{C} - \lambda\mathbf{I}) = (2.5-\lambda)^2 - 4 = 0 \Rightarrow \lambda_1 = 4.5,\ \lambda_2 = 0.5$.
$\boldsymbol{w}_1 = (0.707, 0.707)$, $\boldsymbol{w}_2 = (0.707, -0.707)$. Explained variance: PC1 $\frac{4.5}{5} = \mathbf{90}\%$, PC2 10%.
Project A: $z_1 = \frac{-2-2}{\sqrt{2}} = -2.83$. Keeping only PC1 reduces 2-D to 1-D while keeping 90% of the variance.

Diagram to draw: Principal components of 2-D data



- **Explained variance ratio** $= \lambda_j / \sum \lambda_i$. Choose m so cumulative ratio $\geq 95\%$, or look for the elbow of the **scree plot**. (MNIST: 784 $\rightarrow$ 154 dims keeps 95%.)
- **Reconstruction:** $\hat{\mathbf{X}} = \mathbf{Z}\mathbf{W}_m^\top + \bar{\mathbf{x}}$; reconstruction error = sum of discarded eigenvalues (compression).
- **Randomized PCA** (fast approximation), **Incremental PCA** (mini-batches, large data), **Kernel PCA** (non-linear via kernel trick).

Advantages	reduces dimensions and noise, speeds up learning, removes correlation, helps visualisation, unsupervised
Disadvantages	components hard to interpret, linear only, loses some information, sensitive to scaling and outliers
Applications	image compression, face recognition (eigenfaces), visualisation, gene data, preprocessing for ML

	Feature selection	**Feature extraction (PCA)**
Output	subset of original features	new features (combinations)
Interpretability	high	low
Example	drop irrelevant columns	PCA, LDA

Exam focus

Most asked: curse of dimensionality; projection vs. manifold learning; PCA steps with numerical (covariance, eigenvalues, explained variance); choosing number of components.

Important Questions

- **[2 marks]** Define dimensionality reduction. Give two reasons for using it.
- **[2 marks]** What is the curse of dimensionality?
- **[2 marks]** What is a manifold?
- **[2 marks]** What is the explained variance ratio?
- **[5 marks]** Explain projection and manifold learning.
- **[5 marks]** How do you choose the number of principal components?
- **[5 marks]** Compare feature selection and feature extraction.
- **[10 marks]** Explain PCA with its algorithm and a numerical example.
- **[10 marks]** Explain the curse of dimensionality and the main approaches to dimensionality reduction.

9. Artificial Neural Networks with Keras

Study time: 2.5 hours

9.1 From Biological to Artificial Neurons

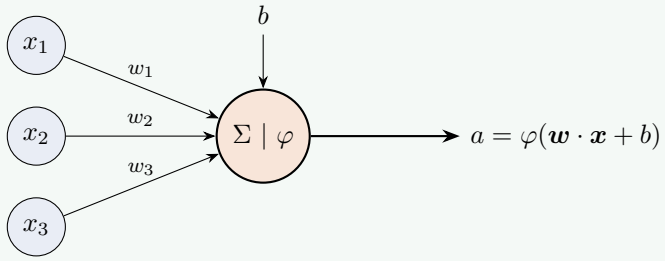
Definition: Artificial neural network (ANN)

An **artificial neural network** is a machine-learning model inspired by the network of biological neurons in the brain. It consists of layers of simple connected units (**artificial neurons**); each connection has a **weight** that is learned from data. ANNs are the core of **deep learning**.

Biological neuron	Artificial neuron
dendrites (receive signals)	inputs x_j
synapse strength	weights w_j
cell body (sums signals)	weighted sum $z = \boldsymbol{w} \cdot \boldsymbol{x} + b$
axon fires if above threshold	activation $a = \varphi(z)$

Why now: huge data, powerful GPUs, better algorithms (ReLU, Adam, dropout), better initialisation.

Diagram to draw: Artificial neuron



9.2 The Perceptron

Definition: Perceptron (Rosenblatt, 1957)

The **perceptron** is the simplest ANN: a single layer of **threshold logic units (TLUs)**. Each TLU computes $z = \boldsymbol{w} \cdot \boldsymbol{x} + b$ and outputs a **step function**: $\hat{y} = 1$ if $z > 0$, else 0. It is a linear binary classifier.

Perceptron learning rule: for each training instance, $w_j \leftarrow w_j + \eta(y - \hat{y})x_j$, $b \leftarrow b + \eta(y - \hat{y})$. Weights change only on mistakes. **Perceptron convergence theorem:** if the data is linearly separable, the rule converges to a solution.

Numerical pattern: perceptron learns AND ($\eta = 1$, start $\boldsymbol{w} = (0, 0)$, $b = 0$)

Epoch 1: $(1, 1)$ predicted 0, $y = 1 \Rightarrow \boldsymbol{w} = (1, 1)$, $b = 1$.
Continuing, after epoch 5: $\boldsymbol{w} = (2, 1)$, $b = -2$; epoch 6 makes no mistakes.
Check: $(1, 1) \to 1 > 0 \Rightarrow 1$; $(1, 0) \to 0 \Rightarrow 0$; $(0, 1) \to -1 \Rightarrow 0$; $(0, 0) \to -2 \Rightarrow 0$ ✓. Boundary $2x_1 + x_2 = 2$.

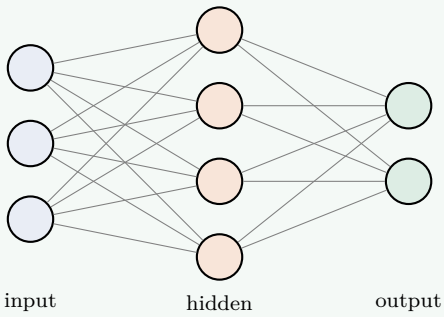
- **XOR problem** (Minsky & Papert, 1969): XOR is not linearly separable, so a single perceptron cannot learn it. Solution: **stack** perceptrons ⇒ multilayer perceptron. E.g. XOR = AND(OR, NAND).

9.3 Multilayer Perceptron (MLP) and Backpropagation

Definition: Multilayer perceptron

An **MLP** consists of an **input layer**, one or more **hidden layers** of neurons, and an **output layer**; every neuron is connected to all neurons of the next layer (**dense / fully connected**). Signals flow only forward (**feedforward network**). An ANN with many hidden layers is a **deep neural network (DNN)**.

Diagram to draw: MLP with one hidden layer



Numerical pattern: forward pass of a 2–2–1 network (sigmoid)

$\boldsymbol{x} = (1, 0.5)$; hidden: $z_1 = 0.4(1) - 0.6(0.5) + 0.1 = 0.2$, $a_1 = \sigma(0.2) = 0.5498$; $z_2 = 0.7(1) + 0.2(0.5) - 0.2 = 0.6$, $a_2 = 0.6457$.
Output: $z = 0.5(0.5498) - 0.3(0.6457) + 0.05 = 0.1312$, $\hat{y} = \sigma(0.1312) = \mathbf{0.533}$. Parameters $= (2 \cdot 2 + 2) + (2 + 1) = 9$.

Parameters of a dense layer $= n_{\text{in}} n_{\text{out}} + n_{\text{out}}$. MLP 784–300–100–10: $235\,500 + 30\,100 + 1\,010 = \mathbf{266\,610}$.

Definition: Backpropagation

Backpropagation is the algorithm used to train MLPs. For each mini-batch it makes a **forward pass** (compute outputs and loss), then a **backward pass** that uses the **chain rule** to compute the gradient of the loss with respect to every weight, layer by layer from output to input, and finally a **gradient descent step** to update the weights.

Steps:

1. Initialise weights randomly (not zero: symmetry breaking).
2. Forward pass: compute every layer's output and the loss.
3. Output error: $\delta^{(L)} = \frac{\partial J}{\partial a^{(L)}} \odot \varphi'(z^{(L)})$.
4. Propagate back: $\delta^{(l)} = (\mathbf{W}^{(l+1)\top} \delta^{(l+1)}) \odot \varphi'(z^{(l)})$.
5. Gradients: $\frac{\partial J}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} a^{(l-1)\top}$; update $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J}{\partial \mathbf{W}}$.
6. Repeat for all mini-batches; one pass over the training set is an **epoch**.

Step function has zero gradient, so MLPs use smooth activations.

9.4 Activation Functions

Function	Formula	Range	Note
Sigmoid	$\frac{1}{1+e^{-z}}$	$(0, 1)$	binary output; vanishing gradient
Tanh	$\frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$(-1, 1)$	zero-centred
ReLU	$\max(0, z)$	$[0, \infty)$	default for hidden layers; fast; "dying ReLU"
Leaky ReLU	$\max(\alpha z, z)$	$(-\infty, \infty)$	fixes dying ReLU
Softmax	$\frac{e^{z_k}}{\sum_j e^{z_j}}$	$(0, 1)$, sum 1	multi-class output

Why non-linear? Without non-linear activations, many layers collapse into a single linear transformation and cannot learn complex patterns.

Task	Output neurons	Output activation	Loss
Regression	1 per value	none (linear)	MSE (or MAE, Huber)
Binary classification	1	sigmoid	binary cross-entropy
Multi-label	1 per label	sigmoid	binary cross-entropy
Multi-class	1 per class	softmax	(sparse) categorical cross-entropy

9.5 Implementing MLPs with Keras

Definition: Keras

Keras is a high-level deep-learning API (part of TensorFlow as `tf.keras`) for building, training, evaluating and running neural networks easily.

Workflow (5 steps):

1. **Load and prepare data:** split train/validation/test; scale inputs (e.g. pixels /255).
2. **Build the model:** `Sequential` API (stack of layers: `Flatten`, `Dense(300, activation="relu")`, ..., `Dense(10, activation="softmax")`).
3. **Compile:** choose loss (`sparse_categorical_crossentropy`), optimizer (`sgd`, `adam`) and metrics (`accuracy`).
4. **Train:** `fit(X, y, epochs, batch_size, validation_data)`; returns a `History` (learning curves).
5. **Evaluate and predict:** `evaluate()` on the test set, `predict()` for new data; `save()` the model.

- **Three APIs: Sequential** (simple stack), **Functional** (multiple inputs/outputs, wide-and-deep networks), **Subclassing** (fully dynamic models).
- **Callbacks:** `ModelCheckpoint` (save best model), `EarlyStopping` (stop when validation loss stops improving), `TensorBoard` (visualise).
- Updates per epoch $= \lceil n/\text{batch size} \rceil$; e.g. $55\,000/32 \to 1719$.
- Example: Fashion-MNIST MLP reaches about 88% test accuracy.

9.6 Training Deep Networks: Key Ideas

- **Hyperparameters:** number of hidden layers, neurons per layer, learning rate (most important), batch size, optimizer, activation, epochs. Tune with validation data or random search (Keras Tuner).
- **Vanishing/exploding gradients:** gradients shrink or grow through many layers. Fixes: ReLU, He/Glorot initialisation, batch normalisation, gradient clipping.
- **Optimizers:** SGD, momentum ($\boldsymbol{v} \leftarrow \beta \boldsymbol{v} - \eta \boldsymbol{g}$), RMSProp, **Adam** (momentum + adaptive rates; default choice).
- **Regularisation (avoid overfitting):** L1/L2 weight penalty, **dropout** (randomly switch off neurons during training, e.g. 20–50%), **early stopping**, more data / data augmentation.

Advantages	learns complex non-linear patterns, automatic feature learning, scales with data and GPUs, state of the art for images, speech and text
Disadvantages	needs large data and compute, black box, many hyperparameters, can overfit
Applications	image recognition, speech recognition, translation, recommendation, self-driving cars, medical imaging

Exam focus

Most asked: perceptron and its learning rule; XOR problem; MLP architecture; backpropagation steps; activation functions table; Keras workflow (build–compile–fit–evaluate); dropout and early stopping.

Important Questions

- **[2 marks]** What is a perceptron?
- **[2 marks]** Why can a perceptron not solve XOR?
- **[2 marks]** Define an activation function. Why is ReLU popular?
- **[2 marks]** What is an epoch?
- **[5 marks]** Compare biological and artificial neurons.
- **[5 marks]** Explain activation functions with formulas.
- **[5 marks]** Explain the Sequential and Functional APIs of Keras.
- **[5 marks]** What is dropout? How does it reduce overfitting?
- **[10 marks]** Explain the multilayer perceptron and the backpropagation algorithm.
- **[10 marks]** Explain the steps to build, compile, train and evaluate an MLP classifier using Keras.

10. TensorFlow: Loading and Preprocessing Data

Study time: 1.5 hours

10.1 TensorFlow

Definition: TensorFlow

TensorFlow is an open-source library by Google for numerical computation, especially suited to large-scale machine learning. Its core is like NumPy with **GPU/TPU support**, **automatic differentiation**, **computation graphs** and **distributed** computing.

Features: runs on CPU, GPU, TPU; mobile (TF Lite), browser (TensorFlow.js), production serving (TF Serving); high-level API Keras; data API `tf.data`; TensorBoard visualisation.

Architecture (top to bottom): Keras / `tf.data` (high-level) → low-level Python API → C++ execution engine (kernels) → CPU / GPU / TPU.

Definition: Tensor

A **tensor** is a multi-dimensional array of values with a fixed **data type** (`dtype`) and **shape**. Rank 0 = scalar, rank 1 = vector, rank 2 = matrix, rank 3+ = higher-order tensor.

- `tf.constant` = immutable tensor; `tf.Variable` = mutable (model weights), changed with `assign()`.
- Operations: `tf.add`, `tf.matmul`, `tf.reduce_mean`, `tf.square`, `tf.reshape`. NumPy arrays convert to tensors and back.
- **No automatic type conversion** (float + int is an error); use `tf.cast`.
- **Broadcasting:** $[[1,2],[3,4]] + [10,20] = [[11,22],[13,24]]$.
- Other structures: sparse tensors, ragged tensors, string tensors, tensor arrays, sets, queues.
- Elements of a tensor = product of its shape: $(32,28,28,1) \to 25\,088$.

Definition: Automatic differentiation (autodiff)

TensorFlow records operations on a `tf.GradientTape` and then computes the **exact gradients** of a result with respect to variables (reverse-mode autodiff). This is how backpropagation is implemented.

Definition: TF functions and graphs

`@tf.function` converts a Python function into a **computation graph** (via AutoGraph and tracing). Graphs are optimised (pruned, parallelised), run faster and can be exported (e.g. 1.45× speed-up in the textbook example).

Customisation: custom loss functions, metrics, layers, models and training loops can be written with the low-level API.

10.2 Loading Data: the `tf.data` API

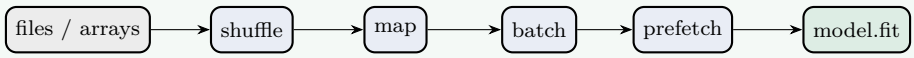
Definition: `tf.data` API

The **`tf.data` API** represents a dataset as a sequence of elements (`tf.data.Dataset`) and provides chainable transformations to **load, transform, shuffle, batch and prefetch** data efficiently, even when it does not fit in memory.

Method	Purpose
`from_tensor_slices()`	create a dataset from arrays/tensors (one element per row)
`map(f)`	apply preprocessing to each element (parallel with `num_parallel_calls`)
`filter(f)`	keep elements satisfying a condition
`shuffle(buffer)`	shuffle using a buffer; do it **before** batching
`batch(n)`	group elements into batches (`drop_remainder` option)
`repeat(k)`	repeat the dataset k times (epochs)
`cache()`	keep data in memory after first epoch
`prefetch(1)`	prepare the next batch while the current one trains (CPU–GPU overlap)
`interleave()`	read several files in parallel

Standard pipeline order: files, `interleave`, `shuffle`, `map` (parse and preprocess), `batch`, `prefetch`, and finally `model.fit(dataset)`.

Diagram to draw: Data pipeline



Numerical pattern: number of batches

$n = 1000$, batch size 32: $\lceil 1000/32 \rceil = \lceil 31.25 \rceil =$ **32** batches (31 full + one of 8).
With `drop_remainder=True`: 31 batches.

10.2.1 The TFRecord format

Definition: TFRecord

TFRecord is TensorFlow's preferred **binary** format for storing large data: a sequence of binary records, usually serialised **protocol buffers** (`tf.train.Example` with features of type bytes, float or int64). Read with `TFRecordDataset`; can be compressed (GZIP).

Advantages: fast to read, compact, supports any data (images, text), works well with `tf.data`.

10.3 Preprocessing Data

Where to preprocess: (1) beforehand (NumPy/Pandas), (2) in the `tf.data` pipeline with `map`, (3) with **Keras preprocessing layers** inside the model, so the same preprocessing is used in training and production (no **training/serving skew**).

Layer	What it does
`Normalization`	standardise: $x' = (x - \mu)/\sigma$; μ, σ learned by `adapt()` on training data
`Discretization`	convert numbers into bins (categories)
`StringLookup`, `IntegerLookup`	map categories to integer indices (vocabulary, OOV buckets)
`CategoryEncoding`	one-hot / multi-hot encoding
`Hashing`	hash categories into a fixed number of bins
`Embedding`	dense trainable vector for each category (#categories × dim parameters)
`TextVectorization`	split text into words and map to indices (or TF-IDF)
`Rescaling`, `Resizing`, `CenterCrop`	image preprocessing; `RandomFlip`, `RandomRotation` for augmentation

Definition: One-hot encoding vs. embedding

One-hot encoding represents a category with K categories as a vector of length K with a single 1 (good for few categories). An **embedding** represents each category by a small **dense, trainable vector**; similar categories get similar vectors (good for many categories, e.g. words).

Numerical pattern: preprocessing

Normalization of $[2, 4, 6]$: $\mu = 4$, $\sigma = \sqrt{8/3} = 1.633 \Rightarrow [-1.225,\ 0,\ 1.225]$.
One-hot with categories (red, green, blue): green $\rightarrow [0, 1, 0]$.
Embedding 50 categories, dimension 8: $50 \times 8 = \mathbf{400}$ parameters.

- **TensorFlow Datasets (TFDS):** ready-made datasets (MNIST, IMDb, ...) loaded with the function `tfds.load()`.
- **TF Transform:** preprocessing for production pipelines (TFX).

Advantages	fast, scalable pipelines, GPU/TPU and distributed support, same preprocessing in training and deployment, production tools
Disadvantages	steeper learning curve than Keras alone, debugging graphs is harder, verbose low-level API
Applications	training on huge image/text datasets, mobile and web deployment, recommendation systems, research

Exam focus

Most asked: what is TensorFlow and its features; tensors and variables; `tf.data` methods and pipeline order; TFRecord; Keras preprocessing layers; one-hot vs. embedding.

Important Questions

- **[2 marks]** What is a tensor? Give its rank examples.
- **[2 marks]** Differentiate `tf.constant` and `tf.Variable`.
- **[2 marks]** What is prefetching?
- **[2 marks]** What is a TFRecord file?
- **[5 marks]** Explain automatic differentiation using `GradientTape`.
- **[5 marks]** Explain the main methods of the `tf.data` API.
- **[5 marks]** Compare one-hot encoding and embeddings.
- **[10 marks]** Explain TensorFlow: architecture, tensors, operations and features.
- **[10 marks]** Explain how data is loaded and preprocessed in TensorFlow using `tf.data`, TFRecord and Keras preprocessing layers.

Last-Hour Revision Sheet

1. Formulas you must know

Least squares	$\theta_1 = \frac{\sum (x-\bar{x})(y-\bar{y})}{\sum (x-\bar{x})^2}$, $\theta_0 = \bar{y} - \theta_1 \bar{x}$, $R^2 = 1 - \frac{\text{SSE}}{\text{SST}}$
Gradient descent	$\theta \leftarrow \theta - \alpha\, \partial J / \partial \theta$
Losses	MSE $\frac{1}{n}\sum (y-\hat{y})^2$; MAE $\frac{1}{n}\sum \lvert y-\hat{y}\rvert$; log-loss $-[y \ln p + (1-y)\ln(1-p)]$
Bias–variance	$\mathbb{E}[(y-\hat{f})^2] = \text{Bias}^2 + \text{Var} + \sigma^2$
Metrics	Acc $\frac{TP+TN}{\text{all}}$, P $\frac{TP}{TP+FP}$, R $\frac{TP}{TP+FN}$, $F_1 = \frac{2PR}{P+R}$
Standard error / CI	$\sigma/\sqrt{n}$; $\bar{x} \pm 1.96\, s/\sqrt{n}$
Distances	Euclidean $\sqrt{\sum (a-b)^2}$, Manhattan $\sum \lvert a-b\rvert$, cosine $\frac{\boldsymbol{a}\cdot\boldsymbol{b}}{\lVert\boldsymbol{a}\rVert\lVert\boldsymbol{b}\rVert}$
Entropy / gain / Gini	$-\sum p \log_2 p$; $H(S) - \sum \frac{\lvert S_v\rvert}{\lvert S\rvert} H(S_v)$; $1 - \sum p^2$
Bayes / Naive Bayes	$P(C\lvert X) = \frac{P(X\lvert C)P(C)}{P(X)}$; $\hat{y} = \arg\max_c P(c) \prod P(x_j\lvert c)$; Laplace $\frac{n+1}{N+V}$
Sigmoid / softmax	$\frac{1}{1+e^{-z}}$; $\frac{e^{z_k}}{\sum e^{z_j}}$; odds $\frac{p}{1-p}$
SVM	margin $\frac{2}{\lVert\boldsymbol{w}\rVert}$; soft margin $\frac{1}{2}\lVert\boldsymbol{w}\rVert^2 + C\sum \xi_i$; hinge $\max(0, 1-yf)$
Kernels	poly $(\gamma \boldsymbol{a}\cdot\boldsymbol{b} + r)^d$; RBF $e^{-\gamma\lVert\boldsymbol{a}-\boldsymbol{b}\rVert^2}$; SVR $\max(0, \lvert y-f\rvert - \varepsilon)$
AdaBoost	$\alpha = \frac{1}{2}\ln\frac{1-\varepsilon}{\varepsilon}$; $w_i \leftarrow w_i e^{-\alpha y_i h(\boldsymbol{x}_i)}$, normalise
OOB	$(1-1/n)^n \approx 36.8\%$ left out
K-Means / silhouette	$J = \sum \lVert\boldsymbol{x} - \boldsymbol{\mu}\rVert^2$; $s = \frac{b-a}{\max(a,b)}$
GMM (E-step)	$\gamma_{ij} = \frac{\pi_j \mathcal{N}_j(\boldsymbol{x}_i)}{\sum_l \pi_l \mathcal{N}_l(\boldsymbol{x}_i)}$
PCA	$\mathbf{C} = \frac{1}{n-1}\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$, $\mathbf{C}\boldsymbol{w} = \lambda\boldsymbol{w}$, explained $= \lambda_j / \sum\lambda$
Neuron / perceptron	$a = \varphi(\boldsymbol{w}\cdot\boldsymbol{x} + b)$; $w \leftarrow w + \eta(y-\hat{y})x$
Dense layer parameters	$n_{\text{in}} n_{\text{out}} + n_{\text{out}}$

2. Ten comparisons asked again and again

Pair	First	Second
Supervised / unsupervised	labelled data, predict y	unlabelled, find structure
Batch / online	all data at once, retrain	incremental, streaming data
Instance / model-based	memorise + similarity (k-NN)	fit parameters (regression)
Underfit / overfit	high bias, too simple	high variance, too complex
Parameter / hyperparameter	learned from data (weights)	set before training (k, C, η)
Linear / logistic regression	number, MSE, closed form	probability, sigmoid, log-loss
Bagging / boosting	parallel, reduces variance	sequential, reduces bias
Hard / soft margin	no violations, outlier-sensitive	slack ξ, controlled by C
K-Means / DBSCAN	needs k, spherical	density, any shape, finds noise
Feature selection / extraction	subset of features	new features (PCA)

3. Which algorithm? One line each

k-NN	majority vote of k nearest points; lazy; scale features
Decision tree	split on highest information gain (ID3) or Gini (CART); prune to avoid overfit
Naive Bayes	Bayes' theorem with independent features; smoothing; fast for text
Linear regression	least-squares line; R^2; normal equation or GD
Logistic regression	sigmoid of a linear score; log-loss; probability output
GLM	distribution + linear predictor + link (identity, logit, log)
SVM	widest street; support vectors; kernels for non-linear; C, γ
Random forest	bagged trees + random features; vote
AdaBoost	reweight mistakes; weighted vote of stumps
Gradient boosting	each tree fits residuals; shrinkage η
Stacking	meta-learner combines base predictions
K-Means	assign to nearest centroid, update means; elbow/silhouette
DBSCAN	core (MinPts in ε), border, noise
GMM	mixture of Gaussians; EM; soft clusters
PCA	eigenvectors of covariance; keep 95% variance
MLP	layers of neurons; backpropagation; ReLU, softmax

4. Mock test (3 hours, 70 marks)

Section A (answer all, $7 \times 2 = 14$)

1. Define machine learning using T, P, E.
2. What is the bias–variance trade-off?
3. Define information gain.
4. What are support vectors?
5. What is out-of-bag evaluation?
6. What is the curse of dimensionality?
7. Why are activation functions needed in an MLP?

Section B (any four, $4 \times 7 = 28$)

8. Explain types of ML systems with examples.
9. Explain k-fold cross-validation and compute $\text{CV}_{(5)}$ for fold errors $0.10, 0.12, 0.08, 0.11, 0.09$. *(Ans. 0.10)*
10. Find the k-NN prediction ($k = 3$) for $(5, 4)$ using the data of Chapter 3.
11. Compare bagging and boosting.
12. Explain DBSCAN with core, border and noise points.
13. Explain the `tf.data` pipeline.

Section C (any two, $2 \times 14 = 28$)

14. Fit $\hat{y} = \theta_0 + \theta_1 x$ to $x = (1, 2, 3, 4, 5)$, $y = (35, 45, 50, 65, 70)$; predict $x = 6$; compute R^2. *(Ans. $26 + 9x$, 80, 0.976)*
15. Explain SVM classification (linear and kernel) and SVR.
16. Apply PCA to $(1, 1), (2, 3), (3, 2), (4, 5), (5, 4)$ and find the explained variance of PC1. *(Ans. 90%)*
17. Explain the MLP, backpropagation and the Keras workflow.

★ Exam focus

Night before the exam: read only the definition boxes, this sheet and the important questions. In the exam: definition first, diagram second, points third, numerical/example fourth, advantages–disadvantages–applications last.